

Optimal Reverse Supply Chain:

Modeling Multi-Echelon, Multi-Indenture Systems

Professor: Amanda Smith

Project Members: Seth Ockerman and Mitchell Stachowiak

University of Wisconsin - Madison

Spring 2026

Execution Guide

We include all the necessary data and figures in the `project.zip` file to run this notebook without there being any extra dependencies. Simply unzip the files, and then run the notebook from top to bottom.

If you wish to recreate our process, the first step is to recreate the input data. To do so, call the bash script that generates the data for each level: `bash gen_data.sh` .

Afterwards, run all of the Julia code from top to bottom. If you run the markdown, the visualizations will fail to load because they are generated in a later step (unless you have them leftover from the original `project.zip`). To generate the visualizations, simply execute `bash gen_output_vis.sh` .

Package Managment

To run the optimization models, as well as to read, write, and present the data, we import all required packages.

As our models will involve the use of many integer variables and constraints, we have chosen to use the commercial optimizer *Gurobi* [2] in conjunction with the Julia optimization framework JuMP.

```
In [1]: # Include ALL Required Packages
import Pkg
Pkg.add(["JuMP", "Gurobi"], io=devnull) # Optimization
Pkg.add(["DataFrames", "CSV"], io=devnull) # I/O
Pkg.add(["Images", "Plots"], io=devnull) # Figures

# Gurobi File Locations & Build if Needed
# ENV["GUROBI_HOME"] = "C:\\Program Files\\gurobi1301\\win64"
# Pkg.build("Gurobi", io=devnull)

# Use ALL Required Packages
using JuMP, Gurobi # Optimization
using DataFrames, CSV # I/O
using Images, Plots # Figures
```

Introduction

The project seeks to create a model that fulfills the demands of a reverse supply chain that is multi-echelon and multi-indenture using linear and integer optimization. Restated more concretely, this project develops a model to meet repair demand for aircraft that are composed of multiple components (e.g. engines, wheels) that each have their own sub-components (e.g. pistons, axles), where each sub-component also consists of smaller parts (e.g., screws). Our solution and report is organized into a series of progressive levels, beginning at level 0, where each increasing level systematically removes simplifying assumptions and introduces greater complexity.

Before we proceed, we define several key terms that will be used throughout this report.

Term	Definition
Reverse Supply Chain	A system of processes that moves goods from the customer back to the producer, often enabling refurbishment, reuse, or redistribution, before the goods are moved back to customers. Examples include refurbishment networks and product return systems.
Multi-echelon	A supply chain structure with multiple interconnected layers (e.g., supplier, warehouses, central hubs) through which goods flow before reaching the customer.
Multi-indenture	A hierarchical product structure composed of multiple levels of components and sub-components (e.g., assemblies, subassemblies, and parts). Decisions such as maintenance, repair, or inventory management must account for these dependencies across levels.
Line Replaceable Unit (LRU) [1]	A self-contained unit that can be replaced upon failure to restore system functionality (e.g. engine) such as on a flight line. LRUs represent the highest level in the multi-indenture system.
Shop Replaceable Unit (SRU) [1]	A sub-component of an LRU that is repaired or replaced (e.g. pistons, battery) in a maintenance facility/shop, sitting one level below LRUs in the multi-indenture hierarchy.

Problem Description

Our project considers aircraft repair logistics as a motivating example. Specifically, we study how to optimally repair aircraft using LRUs composed of multiple distinct SRUs, each of which is built from lower-level parts. These lower-level parts may be shared across SRUs or specialized to a single SRU, with specialized parts stocked only at designated locations. Below, we provide a simplified example of a multi-indenture product structure for clarity.

* marks a shared final-level part. [S] marks a specialized-only part.

```
LRU_1
|-- SRU_1_1 x2
  |-- PART_6* x2
  `-- PART_13* x2
`-- SRU_1_2 x2
  |-- PART_7* x2
  |-- PART_8*[S] x3
```

Repairs occur within a fully connected graph network (i.e. nodes which are connected via edges) that exists in a standard Euclidean grid system. We utilize the following node types: customers, local repair shops, regional warehouses, central depots, and specialized shops. For simplicity, we divide our graph's x-axis into bands which proceed from left to right. We place customers in band 0, local shops in band 1, regional warehouses in band 3, central depots in band 4, and finally, specialized shops in band 5. The majority of our problem assumptions are embedded within the differences between each node type, and as such, we describe each in-detail below.

Customers

Customer nodes generate demand for top-level LRUs in the reverse supply chain, with a hard-coded minimum demand of one item. Customers are connected only to nearby local shops, encoding the assumption that end users do not interact directly with warehouses, depots, or specialized facilities. In later model levels, we introduce the concept of time-slices, where customers have different demands for each time slice.

Local Shop

Local shops are small, nearby facilities that are directly connected to customers. They almost never carry LRUs, occasionally carry SRUs, and usually carry small quantities of low-level parts. By placing local shops between the customer and the regional warehouse, we encode the assumption that local shops serve as access points for intake and light service, rather than broad inventory hubs or high-end repair centers. Local shops have lower repair skill and modest cost, however, they complete repairs quicker than all but specialized shops.

Regional Warehouses

Regional warehouses stand between local shops and central depots or specialized shops. Our scripts assume they have moderate-to-high inventory breadth and depth, they commonly carry SRUs and parts, and occasionally carry LRUs. They have modest repair skill, cost, and time.

Central Depot

Depots are the hubs of our network and have the largest and most diverse inventory. They contain a variety of SRUs and some LRUs, along with a wide range of basic parts. They are connected to regional warehouses and specialized shops. They are the cheapest option and have high skill, however, they are the slowest to complete repairs.

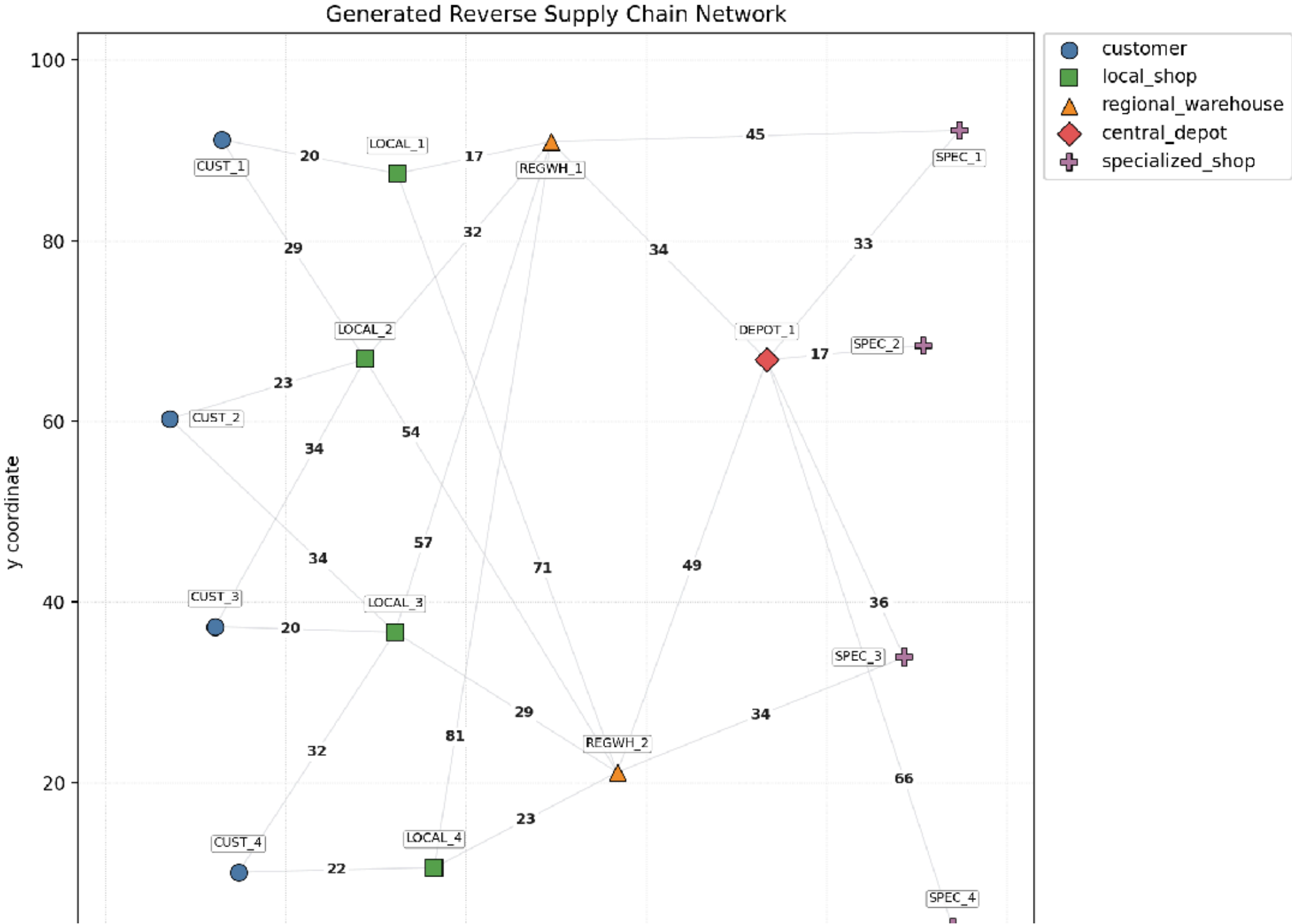
Specialized Shops

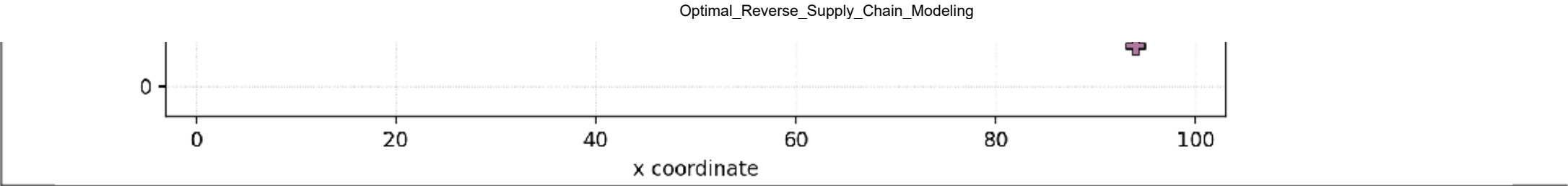
Specialized shops stock fewer items overall, but they frequently stock SRUs and shared parts. Additionally, they contain specialized parts that can only be stocked at these locations. Specialized shops have the highest repair skill and complete repairs the fastest. However, their repairs are the most expensive.

To make our problem structure concrete, we include a visualization of the network used for level 0 of this report. Note that all the level's graphs are very similar, but the specific placement of the nodes varies slightly.

```
In [2]: plot(load("./level0data/network_visualization_graph.png"), axis=nothing, size=(1200, 900))
```

Out[2]:





In summary, we model aircraft reverse supply chain operations with multi-echelon and multi-indenture repairs as a network cost flow problem, combining integer and linear optimization. Our model captures both flow decisions across the network and repair dependencies across indenture levels, which further constrain repair by component availability. The goal of the model is to determine how to route and allocate repairs across the network, so that aircraft repair demand is fulfilled, while minimizing cost and later time metrics and respecting network constraints and component dependencies.

Historical Context and Problem Relevance

This project was inspired by the work of Craig C. Sherbrooke; in particular his work in developing marginal analysis tools such as the METRIC, as well as his textbook *Optimal Inventory Modeling of Systems: Multi-Echelon Techniques* [1]; however, we implement our models using linear programming methods rather than Sherbrooke's marginal approach. Moreover, while his work considered defense applications, our work generalizes to problems faced in the commercial sector such as commercial aviation or trucking, where the same low demand and high cost assumptions can be applied and where reverse supply chains are commonly found. In these areas, it is critical to prevent misallocation of demands and resources as turnarounds are usually slow, mistakes are costly, and inefficiencies can easily compound into other sectors (e.g. commerce, travel, defense).

Our project uses aircraft repair as a motivating example. Because aircraft are far less numerous than automobiles, a single out-of-service aircraft can have a disproportionately large impact on the economy and the flow of travel. At the same time, aircraft are highly specialized systems, and repairs are often both expensive and dependent on specialized parts. This creates a fundamental tradeoff: minimizing downtime may require costly or expedited repairs, while minimizing cost can prolong periods of inactivity. In some cases, repairing a more complex or expensive set of components may return an aircraft to service sooner than lower-cost alternatives that would otherwise be prioritized. This tradeoff makes the problem well-suited for optimization. By adjusting the relative weights of time and cost in the objective function, we can model and evaluate different repair and logistics strategies.

Data Generation

This project uses a parameterized data generator to construct the network, costs, inventory levels, and other problem attributes using realistic (and adjustable) assumptions. While the generator is not the primary focus of this report, it is designed to be flexible so that others can tailor the data to their specific use cases. Through simple command-line parameters, users can control key aspects of the problem, such as the number of customers, the number and types of nodes (e.g., central depots and local shops), and the number of components used to construct LRUs and SRUs. We also provide visualization scripts that allow users to inspect both the generated network and resulting solutions, enabling

intuitive analysis of the system behavior that we use throughout this report. To encode our assumptions in the data generator process, we introduce a set of parameters that control the likelihood of different node attributes. The below table summarizes the key values associated with each node type, and we include a list of simplifying assumptions.

We attempt to keep our data categories to those which would be available to commercial users. As such, it is reasonable that industrial users could instead plug their specific data into the datasets, rather than synthetic generation, to match their specific needs and gain personalized analytic results. In addition, our model does not specifically rely on any specific data generation distribution and so will likely be successful even if our data generation assumptions do not reflect the specific problem faced.

Node Type	Network Role	Inventory	Repair Skill	Repair Capacity	Cost Mult.	Time Mult.	Main Encoded Assumption
Customer	End user / demand source	N/A	N/A	N/A	N/A	N/A	Customers are non-repair, non-inventory endpoints that access the system only through local shops
Local Shop	Intake / light-service node	Mostly parts, tiny SRU presence, effectively no LRU stocking	Low	Low	1.20	0.95	Fairly responsive, but limited in inventory depth, repair scope, and scale
Regional Warehouse	Mid-network generalist	Broad mixed inventory, moderate depth	Low-to-mid	Medium	1.00	1.10	Balanced default option in the middle of the network
Central Depot	Deep stock / high-scale hub	Broadest and deepest inventory, including some LRUs	High	Highest	0.85	3.0	Economies of scale: cheapest and most capable, but slowest
Specialized Shop	Specialized repair node	Focused SRU/part inventory, no LRUs	High	Medium	2.0	0.80	Premium fast specialist: high skill, narrow role, expensive, and necessary for specialized-only parts

Simplifying Assumptions

- Each node has unlimited storage for parts and there is no storage cost.
- There exists a valid solution to fulfill repair demand.
- Arcs have unlimited capacity and shipping cost is equal in both directions.

Outline

This report is organized as follows. As noted above, our approach divides the project into progressive levels, where each increase in level removes simplifying assumptions and introduces additional complexity. Our report present each level of our solution (Levels 0-3.5), with each level including its own "Math Model", "Implementation and Solution" and "Discussion of Results" sections. Note that "Level 0" provides a detailed discussion of how we formulate the repair logistics problem as an optimization model. This foundation is

essential for understanding the subsequent levels, which primarily focus on the changes made relative to earlier model formulations. Finally, the "Conclusion" section summarizes our findings and outlines directions for future work.

Level 0: Math Model, Implementation, Results, and Discussion

Level 0 serves as a minimum viable model that captures the proposed problem in a simplified setting. The Level 0 model captures basic shipping patterns as network flow and allows nodes to perform repairs as needed. Demand is generated by customer nodes, which is then satisfied using available/repaired LRUs. We view this as a minimization problem, where in Level 0, the objective is to minimize the total cost of all required transportation and repairs.

To satisfy demand, the model has two options. If a demanded LRU is available within a node's inventory (see `quantity` in `node_inventory.csv` for initial inventory), the model can transport that LRU through the network to the customer node to fill the demand; doing so incurs a transportation cost of traversing the nodes from the LRU's original node to the customer nodes (i.e., the sum of costs along all traversed arcs). Alternatively, the model can transport subcomponents of the LRU to some repair node, use them to repair the demanded LRU, and then transport that repaired LRU to the customer. This process incurs the transportation cost of the subcomponents to the repair node, repair cost to fix the LRU, and then the final transportation cost of shipping the LRU from the repair center to the customer. The second approach is necessary when a demanded LRU is not present in a node's initial inventory, and it provides an additional option to the model if repairing an LRU at a local shop may be cheaper than shipping a complete LRU from a far node.

For a node to repair an LRU or SRU, the required subcomponents must be available at that node in the necessary quantities. Repair costs and component indenture relationships are specified in `bom.csv`. We model this requirement as a flow constraint (to be discussed in more detail below). After the repair is complete, the required subcomponents are removed from the repair node's inventory, and the finished component is added. For Level 0, any repair node is allowed to perform any repair, regardless of the skill, equipment, or specialization that repair might realistically require. This is useful for establishing a baseline, but it is unrealistic because some repairs require specialized knowledge or equipment. We address this limitation in the next level of the model.

To model realistic dynamics, each repair is assigned an associated cost. Although the exact repair costs are sampled randomly, the data generation process encodes modeling assumptions by specifying different cost ranges for each repair. In general, higher-echelon components, such as LRUs, are cheaper to repair than lower-echelon components, such as SRUs (see `parent_component_cost` in `bom.csv`). This assumption reflects the idea that repairing a higher-echelon component often amounts to replacing subcomponents, where diagnosis may be easier because the component is larger and more modular. In contrast, lower-echelon repairs may require more detailed diagnosis, such as locating a crack or identifying a burned-out capacitor, as well as more specialized modifications, such as welding or soldering, in addition to assembly.

We assign a transportation cost to each arc based on the Euclidean distance between nodes (see `cost` in `arcs.csv`). Transportation costs are symmetric, meaning the cost of traveling from Node 0 to Node 1 is the same as the cost of traveling from Node 1 to Node 0. In all but one of our model levels, transportation cost is charged separately for each

part, SRU, and LRU. This simplifying assumption is unrealistic because items would likely be shipped in batches, with an initial fixed cost for using a truck and a marginal cost associated with added weight, such as reduced fuel efficiency. We partly address this limitation in Level 3.5, however, we leave a more detailed implementation of transportation logistics to future work.

To model component flow and repair throughout the network, we implement a variation of the standard flow-balance constraint used in minimum-cost network flow (MCNF) problems. Specifically, for each node and component type, we track both net gains and net losses. A component's net gain includes the node's initial inventory of that component, any inflow from other nodes, and any repairs that produce the component. A component's net loss includes any units shipped to other nodes and any units consumed as subcomponents in repairs. For customer nodes, net loss also includes demand for the component (this is technically true for all nodes, but only customer nodes have non-zero demand). We require each component's net gain to be at least as large as its net loss at each node. This ensures physical feasibility, i.e. a node cannot ship or consume more units of a component than it initially had, received from other nodes, or produced through repair. Note that, unlike standard MCNF, we do not require exact equality between net gain and net loss. This is because not all components are necessarily consumed or transported to satisfy demand. As a result, nodes may retain leftover inventory after all demands are fulfilled, whereas standard MCNF typically enforces exact flow balance.

The data used to test our model provides sufficient parts and inventory to satisfy all demand through a combination of transportation and repair. As a result, the model is guaranteed to have a feasible solution; however, this guarantee may not hold in the real world if there are not enough parts, inventory, or repair capacity. We defer exploration of a model that operates under scarcity to future work.

Math Model: Level 0

Our goal is to determine how to route components and perform repairs such that we minimize the total cost of fulfilling customer demand.

Problem Data

Let N denote the set of nodes within the network (i.e. customers, shops, warehouses, depots); let A denote the set of arcs connecting these nodes (i.e. roads, shipping routes, flights) that components can be transported through; and let K denote the set of components including LRUs, SRUs, and parts. In addition, let $d_{n,k} \geq 0$ denote the number of component $k \in K$ demanded at node $n \in N$; let $I_{n,k} \geq 0$ denote the number of component $k \in K$ demanded at node $n \in N$; let $K_k \subseteq K$ denote the set of components required to repair $k \in K$ with associated quantities $q_{k,k'} \geq 0$ of the subcomponent $k' \in K_k$; let $C_k^{\text{repair}} \geq 0$ denote the standard cost of repairing component $k \in K$ and $C_a^{\text{flow}} \geq 0$ denote the standard cost of traversing arc $a \in A$.

Decision variables

Let $x_{a,k} \in \mathbb{Z}$ denote the number of component $k \in K$ which are transported through arc $a \in A$; let $r_{n,k} \in \mathbb{Z}$ denote the number of component $k \in K$ which are repaired at node $n \in N$.

We next construct our constraints.

We cannot have negative flow between nodes, nor can we have negative repairs at nodes; thus, we have the nonnegativity constraints,

$$\begin{aligned} x_{a,k} &\geq 0, & a \in A, k \in K \\ r_{n,k} &\geq 0. & n \in N, k \in K \end{aligned}$$

We can only allow components to flow via the allowed pathways of transportation and repair, including their consumption for repair and existence as present inventory. As such, we utilize a slightly modified MCNF flow constraint,

$$\left[I_{n,k} + \sum_{(i,n) \in A} x_{(i,n),k} + r_{n,k} \geq \sum_{(n,i) \in A} x_{(n,i),k} + \sum_{k' \in K_k} r_{n,k'} q_{k',k} + d_{n,k} \right], \quad n \in N, k \in K$$

which represents *inventory* + *inflow* + *repair* = *outflow* + *consumed* + *demand* of a single component at a single node.

Objective Function

We want to minimize the cost of filling demands; thus, we want to minimize the cost associated with components traversing the network, captured as

$$\sum_{a \in A, k \in K} C_a^{flow} x_{a,k},$$

and we want to minimize the cost of all repairs, captured as the sum of all components, SRU, and LRU repairs at each node:

$$\sum_{n \in N, k \in K} C_k^{repair} r_{n,k},$$

together; hence, we have the cost objective function as their sum, which we minimize as,

$$\sum_{a \in A, k \in K} C_a^{flow} x_{a,k} + \sum_{n \in N, k \in K} C_k^{repair} r_{n,k}.$$

Putting this together, we have the integer linear program,

$$\begin{aligned} \min \quad & \sum_{a \in A, k \in K} C_a^{\text{flow}} x_{a,k} + \sum_{n \in N, k \in K} C_k^{\text{repair}} r_{n,k} \\ \text{s.t.} \quad & \left[I_{n,k} + \sum_{(i,n) \in A} x_{(i,n),k} + r_{n,k} \geq \right. \\ & \left. \sum_{(n,i) \in A} x_{(n,i),k} + \sum_{k' \in K_k} r_{n,k'} q_{k',k} + d_{n,k} \right], \quad n \in N, k \in K \\ & x_{a,k} \geq 0, \quad a \in A, k \in K \\ & r_{n,k} \geq 0, \quad n \in N, k \in K \end{aligned}$$

Implementation: Level 0

We load in the Level 0 data from the relevant CSV files and convert the data into more convenient data-structures (e.g., dictionaries and lists) that are then used to implement the math model above using the JuMP framework.

```
In [3]: # Load Data
node_df = CSV.read("level0data/nodes.csv", DataFrame, delim=',')
arc_df = CSV.read("level0data/arcs.csv", DataFrame, delim=',')
bom_df = CSV.read("level0data/bom.csv", DataFrame, delim=',')
demand_df = CSV.read("level0data/demand.csv", DataFrame, delim=',')
node_inv_df = CSV.read("level0data/node_inventory.csv", DataFrame, delim=',')

# Convert node_df to Node Information
#-----
struct Node # Node struct
    name::String
    node_type::String
    loc::Tuple{Float64, Float64}
end
node_dict = Dict() # dictionary of nodes -> Node struct
for df_node in eachrow(node_df) # add nodes to node_dict
    node_id = df_node.node_id
    node_type = df_node.node_type
    (x, y) = (df_node.x, df_node.y)
    node = Node(node_id, node_type, (x, y))
    node_dict[node_id] = node
end
```

```

end
nodes = collect(keys(node_dict)) # List of nodes

# Convert arc_df to Arc Information
#-----
struct Arc # Arc struct
    name::String
    fromto::Tuple{String, String}
    trans_cost::Float64
end
arc_dict = Dict() # dictionary of arcs -> Arc struct
for df_arc in eachrow(arc_df) # add arcs to arc_dict
    arc_id = df_arc.arc_id
    node_from = df_arc.from_node
    node_to = df_arc.to_node
    trans_cost = df_arc.cost
    arc = Arc(arc_id, (node_from, node_to), trans_cost)
    arc_dict[(node_from, node_to)] = arc
end
arcs = collect(keys(arc_dict)) # List of arcs of network
incoming = Dict() # dictionary of node to nodes incident to the node
outgoing = Dict() # dictionary of node to nodes incident from the node
for (node_from, node_to) in arcs # add nodes to incoming and outgoing
    push!(get!(incoming, node_to, []), node_from)
    push!(get!(outgoing, node_from, []), node_to)
end

# Convert bom_df to Bill of Material (BOM) Information
#-----
struct Component # Component struct
    comp_id::String
    comp_type::String
    repair_cost::Float64
    subcomps::Vector{Tuple{String, Int}}
end
BOM = Dict() # dictionary of component -> Component struct
for bom_item in eachrow(bom_df) # build dictionary
    comp_id = bom_item.parent_component_id

```

```

    comp_type = bom_item.parent_component_class
    comp_repair_cost = bom_item.parent_component_cost
    comp_repair_list = []
    sub_bom = subset(bom_df, :parent_component_id => ByRow(==(comp_id)))
    for sub_comp in eachrow(sub_bom)
        sub_comp_id = sub_comp.child_component_id
        sub_comp_qty = sub_comp.quantity_required
        push!(comp_repair_list, (sub_comp_id, sub_comp_qty))
    end
    comp = Component(comp_id, comp_type, comp_repair_cost, comp_repair_list)
    BOM[comp_id] = comp
end
part_bom = subset(bom_df, :child_component_class => ByRow(=="part")) # add parts components to BOM
for bom_item in eachrow(part_bom)
    comp_id = bom_item.child_component_id
    comp_type = "part"
    comp_repair_cost = 0
    comp_repair_list = []
    comp = Component(comp_id, comp_type, comp_repair_cost, comp_repair_list)
    BOM[comp_id] = comp
end
components = collect(keys(BOM)) # list of components in BOM
subcomp_dict = Dict{< i> } # dictionary of comonents -> list of subcomponents
for comp in components
    subcomp_bom = subset(bom_df, :child_component_id => ByRow(==(comp)))
    subcomp_for_lst = []
    for subcomp_for in eachrow(subcomp_bom)
        push!(subcomp_for_lst, (subcomp_for.parent_component_id, subcomp_for.quantity_required))
    end
    subcomp_dict[comp] = subcomp_for_lst
end

# Convert demand_df to Demand Information
#-----
struct Demand # Demand struct
    name::String
    loc::String
    component::String
    qty::Int
end

```

```

demands = Dict() # dictionary of (node, comp) -> Demand struct
for df_demand in eachrow(demand_df) # build dictionary
    demand_id = df_demand.demand_id
    demand_node = df_demand.node_id
    demand_comp = df_demand.component_id
    qty = df_demand.quantity
    demand = Demand(demand_id, demand_node, demand_comp, qty)
    demands[(demand_node, demand_comp)] = demand
end

# Convert node_inv_df to Node Inventory Information
#-----
struct Stock # Stock struct
    loc::String
    component::String
    qty::Int
end
inventory = Dict() # dictionary of (node, comp) -> Demand struct
for df_stock in eachrow(node_inv_df) # build dictionary
    stock_node = df_stock.node_id
    stock_comp = df_stock.component_id
    qty = df_stock.quantity
    stock = Stock(stock_node, stock_comp, qty)
    inventory[(stock_node, stock_comp)] = stock
end

```

We then directly implement the Level 0 math model using these data-structures.

```

In [4]: # Model
m = Model(Gurobi.Optimizer)

# Variables
@variable(m, x[arc in arcs, comp in components] >= 0, Int) # flow of x component through arc
@variable(m, r[node in nodes, comp in components] >= 0, Int) # r repairs of component at node

```

```

# Constraints

## Flow Conservation
for comp in components, node in nodes
    inflow = sum(x[(node_from, node), comp]
                 for node_from in incoming[node])           # flow of component into node
    outflow = sum(x[(node, node_to), comp]
                  for node_to in outgoing[node])             # flow of component out of node
    demand = get(demands, (node, comp), Demand("", "", "", 0)).qty # demand of component present at node
    stock = get(inventory, (node, comp), Stock("", "", 0)).qty # initial stock of component at node
    repairs = r[node, comp] # repairs of component at node
    consumed = sum(qty * r[node, comp_repaired]
                   for (comp_repaired, qty) in subcomp_dict[comp]; init=0) # component consumed for repairs at node
    @constraint(m, stock + inflow + repairs >= outflow + consumed + demand)
end

## Cannot Repair into Part Components
for comp in components
    if BOM[comp].comp_type == "part"
        for node in nodes
            @constraint(m, r[node, comp] == 0)
        end
    end
end

# Objective
@objective(m, Min,
            sum(arc_dict[arc].trans_cost * x[arc, comp] for comp in components, arc in arcs) + # transport cost
            sum(BOM[comp].repair_cost * r[node, comp] for comp in components, node in nodes) # repair cost
            )

# Solve
set_silent(m)
optimize!(m)

```

Set parameter Username

Set parameter LicenseID to value 2807856

Academic license - for non-commercial use only - expires 2027-04-15

Solution: Level 0

```
In [5]: # Output
println("Optimal Objective Value: ", round(objective_value(m), digits=2)) # print objective value
println()

## Flow Output
flow_output_df = DataFrame(comp_id = String[],
                           node_from = String[],
                           node_to = String[],
                           qty = Int[]
                           )
for comp in components, (node_from, node_to) in arcs # store optimal flow in dataframe
    val = value(x[(node_from, node_to), comp])
    if val > 1e-3
        push!(flow_output_df, (comp_id = comp,
                               node_from = node_from,
                               node_to = node_to,
                               qty = val)
        )
    end
end
sort!(flow_output_df, [:comp_id, :node_from, :node_to])
println("Optimal Flow Schedule:") # output optimal flow dataframe
println(flow_output_df)
println()

## Repair Output
repair_output_df = DataFrame(comp_id = String[],
                             node_id = String[],
                             qty = Int[]
                             )
for comp in components, node in nodes # store optimal repair in dataframe
    val = value(r[node, comp])
    if val > 1e-3
        push!(repair_output_df, (comp_id = comp,
```



```
                node_id = node,
                qty = val)
        )
    end
end
sort!(repair_output_df, [:comp_id, :node_id,])
println("Optimal Repair Schedule:") # output optimal repairs
println(repair_output_df)

## Convert Output to CSV
mkpath("level0output")
CSV.write("level0output/flow_output.csv", flow_output_df)
CSV.write("level0output/repair_output.csv", repair_output_df);
```

Optimal Objective Value: 1402.0

Optimal Flow Schedule:

15x4 DataFrame

Row	comp_id	node_from	node_to	qty
	String	String	String	Int64
1	LRU_2	DEPOT_1	REGWH_1	1
2	LRU_2	DEPOT_1	REGWH_2	1
3	LRU_2	LOCAL_2	CUST_2	2
4	LRU_2	LOCAL_3	CUST_3	3
5	LRU_2	LOCAL_4	CUST_4	1
6	LRU_2	REGWH_1	LOCAL_2	2
7	LRU_2	REGWH_2	LOCAL_3	3
8	LRU_2	REGWH_2	LOCAL_4	1
9	LRU_2	SPEC_1	REGWH_1	1
10	LRU_2_2	DEPOT_1	REGWH_2	3
11	LRU_3	LOCAL_2	CUST_3	2
12	LRU_3	REGWH_1	LOCAL_2	2
13	LRU_5	DEPOT_1	REGWH_1	3
14	LRU_5	LOCAL_1	CUST_1	3
15	LRU_5	REGWH_1	LOCAL_1	3

Optimal Repair Schedule:

5x3 DataFrame

Row	comp_id	node_id	qty
	String	String	Int64
1	LRU_2	DEPOT_1	2
2	LRU_2	REGWH_2	3
3	LRU_2	SPEC_1	1
4	LRU_3	REGWH_1	2
5	LRU_5	DEPOT_1	3

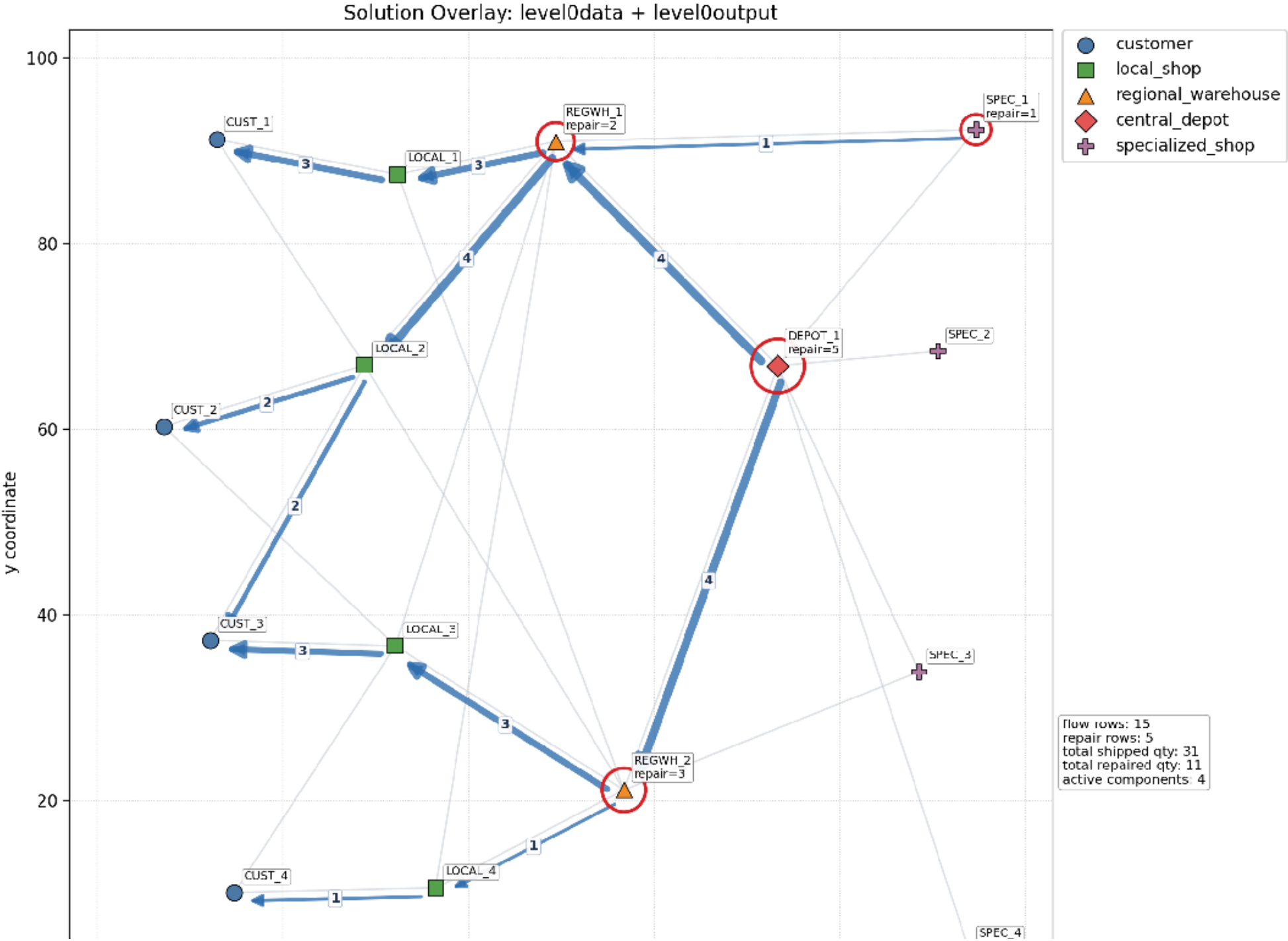
Discussion: Level 0

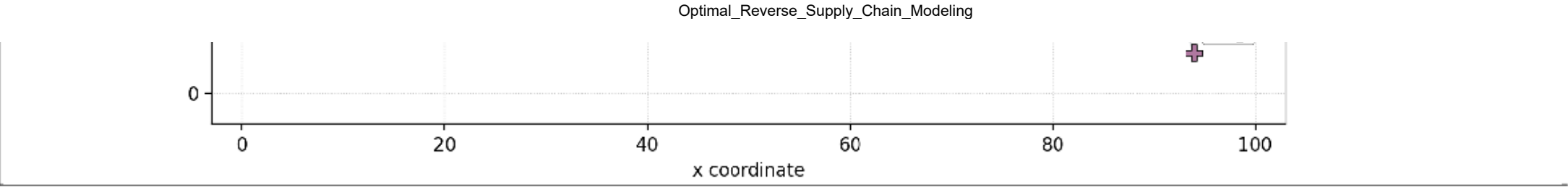
As shown in the figure below, Level 0 produces a relatively simple sequence of events: most repairs are performed at the central depot, and only a single repair is performed at a specialized repair shop. As expected, the model avoids traversing long edges when shorter routes are available, even if doing so requires traveling across multiple intermediate edges.

Initially, we were surprised to see that only LRUs were repaired. However, this behavior is explained by the relative abundance of parts compared to demand in the generated test data. Because sufficient functional SRUs were already available, the model could ship existing SRUs where needed rather than constructing new ones. This is an artifact of the testing data rather than a fundamental feature of the model. This early behavior highlights an important theme that will reoccur throughout this report. While the model formulation determines the feasible decision space, the solution selected in practice is fundamentally influenced by the input data. As a result, the optimizer often selects the cheapest feasible option rather than naturally exhibiting the full complexity of the behavior we aim to capture.

```
In [6]: plot(load("./level0output/solution_network_all.png"), axis=none, size=(1200, 900))
```

Out[6]:

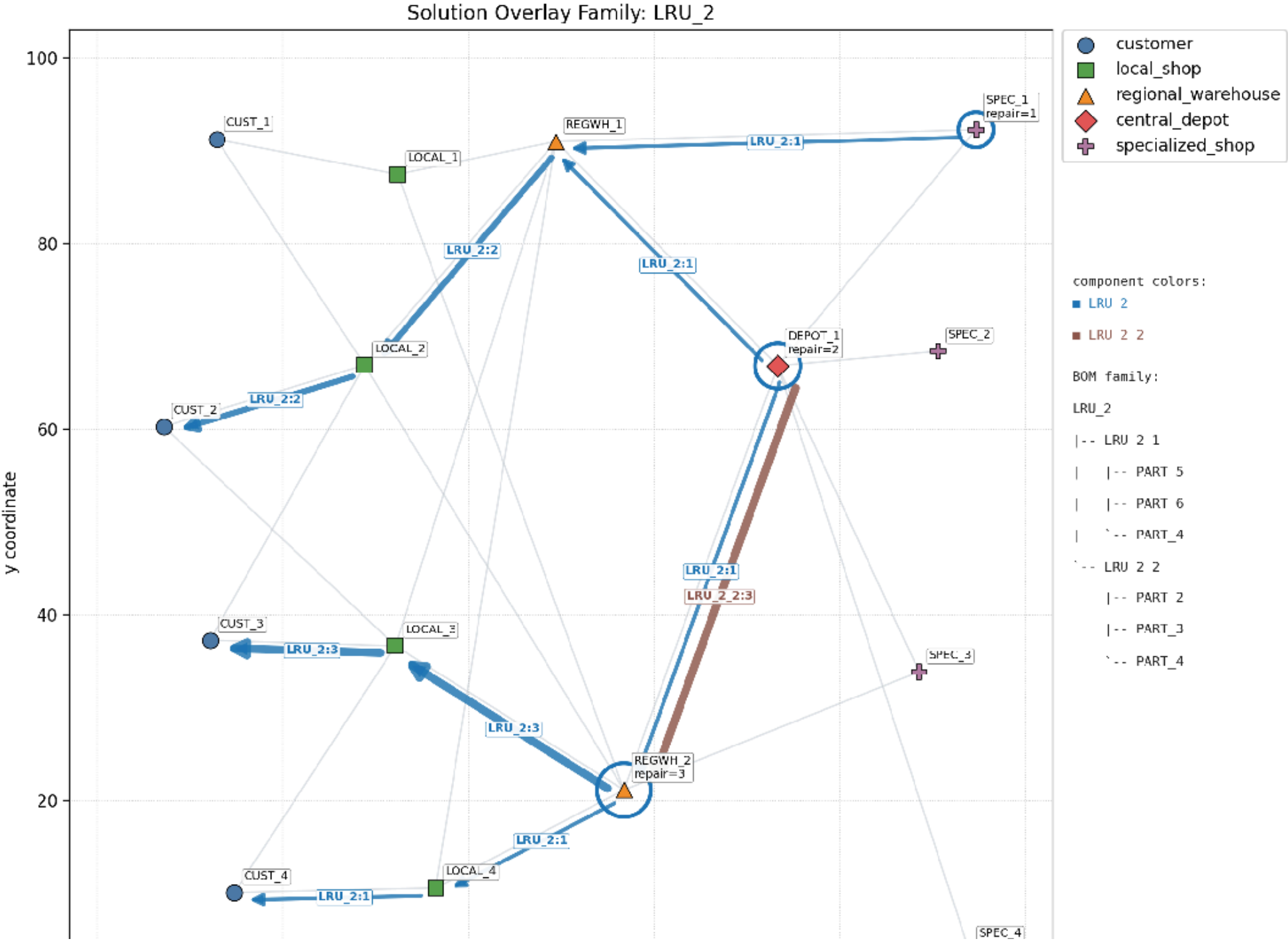


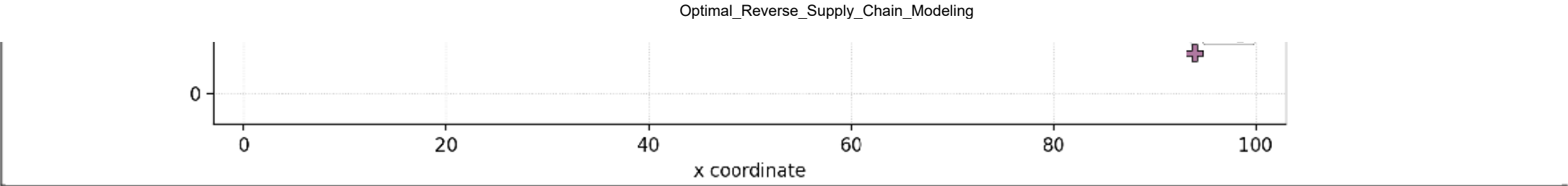


If we zoom in on specifically LRU_2 (shown below), we see that a sub-component SRU (shown as LRU_2_2) is shipped from the depot to REGWH_2, before it is repaired there and shipped to the customer. Per-component repair and movement visualizations are available in `level0output/solution_by_component`.

```
In [7]: plot(load("./level0output/solution_family_lru_2.png"), axis=nothing, size=(1200, 900))
```

Out[7]:





Level 1: Math Model, Implementation, Results, and Discussion

Level 1 improves on Level 0 by refining the repair logic. In particular, the Level 1 model adds repair-cost adjustments by shop type, represented as a node-local multiplier applied to each component’s base repair cost. It also restricts which repairs can be performed at each shop type. Higher-skilled shop types have larger cost multipliers, reflecting the need to hire more specialized technicians and maintain more advanced repair equipment.

Cost multipliers (`repair_cost_multiplier` in `component_multiplier.csv`) are applied directly to the objective function. In effect, this forces the model to prioritize repairs locations with lower multipliers to minimize cost. We assign regional warehouses a baseline cost multiplier of 1.0. Local shops are modeled as slightly more expensive, with a multiplier of 1.2, reflecting the cost of proximity. Central depots are modeled as less expensive, with a multiplier of 0.85, reflecting economies of scale. Specialized shops are modeled as substantially more expensive, with a multiplier of 2.0, reflecting their highly trained technicians and specialized equipment.

In order to perform a repair, we now require the skill of the shop to match or exceed the difficulty of the repair. This captures the idea that some repairs, especially larger or more complex repairs such as LRUs, may be beyond the capabilities of smaller or less experienced repair locations. Our python scripts generate a random value between 1 and 10 that represents the repair difficulty and skill level for repairs and shop types, respectively (see `parent_component_repair_difficulty` in `bom.csv` for repair difficulty; see `shop_capability` in `nodes.csv` for shop skill), with each shop type and repair having a range that its value is sampled from. Depots and specialized shops have the highest skill level, capturing their expertise and equipment availability. Local shops and regional warehouses have more modest skill level because they are less likely to have specialized equipment and highly trained technicians. Higher-echelon repairs, such as final LRU assembly, have higher repair difficulties, reflecting the greater expertise required to assemble unique aircraft components. In contrast, lower-echelon repairs, such as fixing a less specialized SRU component like a piston, are modeled as requiring less repair difficulty.

We expect that our modifications will cause LRU repairs to be completed at specialized shops and depots, while lower echelon components like SRUs will be repaired at a mix of locations. In particular, given that depots have high skill and low cost, we expect the model to complete the majority of repairs at depots.

Math Model: Level 1

We want to modify our Level 0 math model to include the extra logic of Level 1. Specifically, we want to add constraints preventing repairs of difficulty higher than the shop's skill and modify the objective function to account for repair cost multipliers by location.

Additional Problem Data

Let $N_k \subseteq N$ denote the set of nodes within the network for which component $k \in K$ can be repaired. In addition, let $m_n^{\text{cost}} \geq 0$ denote the repair cost multiplier at node $n \in N$.

Additional Decision Variables

N/A

Additional Constraints

We cannot repair components of difficulty greater than the shop's capability,

$$r_{n,k} = 0. \qquad n \in N \setminus N_k, \, k \in K$$

Objective Function Modifications

We apply our repair cost multiplier to the repair cost,

$$\sum_{n \in N, k \in K} m_n^{\text{cost}} C_k^{\text{repair}} r_{n,k};$$

hence, we have the modified cost objective function we want to minimize as,

$$\sum_{a \in A, k \in K} C_a^{\text{flow}} x_{a,k} + \sum_{n \in N, k \in K} m_n^{\text{cost}} C_k^{\text{repair}} r_{n,k}.$$

Putting this together, we have the integer linear program,

$$\begin{aligned} \min \quad & \sum_{a \in A, k \in K} C_a^{\text{flow}} x_{a,k} + \sum_{n \in N, k \in K} m_n^{\text{cost}} C_k^{\text{repair}} r_{n,k} \\ \text{s.t.} \quad & \left[\begin{aligned} I_{n,k} + \sum_{(i,n) \in A} x_{(i,n),k} + r_{n,k} \geq \\ \sum_{(n,i) \in A} x_{(n,i),k} + \sum_{k' \in K_k} r_{n,k'} q_{k',k} + d_{n,k} \end{aligned} \right], \quad n \in N, \, k \in K \\ & r_{n,k} = 0, \quad n \in N \setminus N_k, \, k \in K \\ & x_{a,k} \geq 0, \quad a \in A, \, k \in K \\ & r_{n,k} \geq 0, \quad n \in N, \, k \in K \end{aligned}$$

Implementation: Level 1

We first load in the level 1 data.

```
In [8]: # Load Data
node_df = CSV.read("level1data/nodes.csv", DataFrame,delim=',')
arc_df = CSV.read("level1data/arcs.csv", DataFrame,delim=',')
bom_df = CSV.read("level1data/bom.csv", DataFrame,delim=',')
demand_df = CSV.read("level1data/demand.csv", DataFrame,delim=',')
node_inv_df = CSV.read("level1data/node_inventory.csv", DataFrame,delim=',')
mult_df = CSV.read("level1data/component_multiplier.csv", DataFrame,delim=',')

# Convert node_df to Node Information
#-----
struct Node # Node struct
  name::String
  node_type::String
  loc::Tuple{Float64, Float64}
  repair_capability::Int
end
node_dict = Dict() # dictionary of nodes -> Node struct
for df_node in eachrow(node_df) # add nodes to node_dict
  node_id = df_node.node_id
  node_type = df_node.node_type
```

```

    (x, y) = (df_node.x, df_node.y)
    repair_capability = df_node.repair_capability
    node = Node(node_id, node_type, (x, y), repair_capability)
    node_dict[node_id] = node
end
nodes = collect(keys(node_dict)) # list of nodes

# Convert arc_df to Arc Information
#-----
struct Arc # Arc struct
    name::String
    fromto::Tuple{String, String}
    trans_cost::Float64
end
arc_dict = Dict() # dictionary of arcs -> Arc struct
for df_arc in eachrow(arc_df) # add arcs to arc_dict
    arc_id = df_arc.arc_id
    node_from = df_arc.from_node
    node_to = df_arc.to_node
    trans_cost = df_arc.cost
    arc = Arc(arc_id, (node_from, node_to), trans_cost)
    arc_dict[(node_from, node_to)] = arc
end
arcs = collect(keys(arc_dict)) # List of arcs of network
incoming = Dict() # dictionary of node -> nodes incident to the node
outgoing = Dict() # dictionary of node -> nodes incident from the node
for (node_from, node_to) in arcs # add nodes to incoming and outgoing
    push!(get!(incoming, node_to, []), node_from)
    push!(get!(outgoing, node_from, []), node_to)
end

# Convert bom_df to Bill of Material (BOM) Information
#-----
struct Component # Component struct
    comp_id::String
    comp_type::String
    repair_difficulty::Int
    repair_cost::Float64

```

```

    subcomps::Vector{Tuple{String, Int}}
end
BOM = Dict() # dictionary of component -> Component struct
for bom_item in eachrow(bom_df) # build dictionary
    comp_id = bom_item.parent_component_id
    comp_type = bom_item.parent_component_class
    comp_repair_difficulty = bom_item.parent_component_repair_difficulty
    comp_repair_cost = bom_item.parent_component_cost
    comp_repair_list = []
    sub_bom = subset(bom_df, :parent_component_id => ByRow(==(comp_id)))
    for sub_comp in eachrow(sub_bom)
        sub_comp_id = sub_comp.child_component_id
        sub_comp_qty = sub_comp.quantity_required
        push!(comp_repair_list, (sub_comp_id, sub_comp_qty))
    end
    comp = Component(comp_id, comp_type, comp_repair_difficulty, comp_repair_cost, comp_repair_list)
    BOM[comp_id] = comp
end
part_bom = subset(bom_df, :child_component_class => ByRow(=="part")) # add parts components to BOM
for bom_item in eachrow(part_bom)
    part_id = bom_item.child_component_id
    part_type = "part"
    part_repair_difficulty = 0
    part_repair_cost = 0
    part_repair_list = []
    part = Component(part_id, part_type, part_repair_difficulty, part_repair_cost, part_repair_list)
    BOM[part_id] = part
end
components = collect(keys(BOM)) # list of components in BOM
subcomp_dict = Dict() # dictionary of comonents -> list of subcomponents
for comp in components
    subcomp_bom = subset(bom_df, :child_component_id => ByRow(==(comp)))
    subcomp_for_lst = []
    for subcomp_for in eachrow(subcomp_bom)
        push!(subcomp_for_lst, (subcomp_for.parent_component_id, subcomp_for.quantity_required))
    end
    subcomp_dict[comp] = subcomp_for_lst
end

# Convert demand_df to Demand Information

```

```

#-----
struct Demand # Demand struct
  name::String
  loc::String
  component::String
  qty::Int
end
demands = Dict() # dictionary of (node, comp) -> Demand struct
for df_demand in eachrow(demand_df) # build dictionary
  demand_id = df_demand.demand_id
  demand_node = df_demand.node_id
  demand_comp = df_demand.component_id
  qty = df_demand.quantity
  demand = Demand(demand_id, demand_node, demand_comp, qty)
  demands[(demand_node, demand_comp)] = demand
end

# Convert node_inv_df to Node Inventory Information
#-----
struct Stock # Stock struct
  loc::String
  component::String
  qty::Int
end
inventory = Dict() # dictionary of (node, comp) -> Demand struct
for df_stock in eachrow(node_inv_df) # build dictionary
  stock_node = df_stock.node_id
  stock_comp = df_stock.component_id
  qty = df_stock.quantity
  stock = Stock(stock_node, stock_comp, qty)
  inventory[(stock_node, stock_comp)] = stock
end

# Convert comp_mult_df to Component Multiplier Information
#-----
struct Mults # Mult struct
  loc::String
  component::String

```

```

        repair_cost_mult::Float64
    end
    mult_dict = Dict{<i># dictionary of (node, comp) -> Comp_Mult struct</i>}
    for df_mults in eachrow(mult_df) # build dictionary
        mults_node = df_mults.node_id
        mults_comp = df_mults.component_id
        repair_cost_mult = df_mults.repair_cost_multiplier
        mults = Mults(mults_node, mults_comp, repair_cost_mult)
        mult_dict[(mults_node, mults_comp)] = mults
    end
end

```

We then implement the level 1 model in Julia.

```

In [9]: # Model
m = Model(Gurobi.Optimizer)

# Variables
@variable(m, x[arc in arcs, comp in components] >= 0, Int) # flow of x component through arc
@variable(m, r[node in nodes, comp in components] >= 0, Int) # r repairs of component at node

# Constraints
## Flow Conservation
for comp in components, node in nodes
    inflow = sum(x[(node_from, node), comp]
                 for node_from in incoming[node]) # flow of component into node
    outflow = sum(x[(node, node_to), comp]
                 for node_to in outgoing[node]) # flow of component out of node
    demand = get(demands, (node, comp), Demand("", "", "", 0)).qty # demand of component present at node
    stock = get(inventory, (node, comp), Stock("", "", 0)).qty # initial stock of component at node
    repairs = r[node, comp] # repairs of component at node
    consumed = sum(qty * r[node, comp_repaired]
                  for (comp_repaired, qty) in subcomp_dict[comp]; init=0) # component consumed for repairs at node
    @constraint(m, stock + inflow + repairs >= outflow + consumed + demand)
end

```

```

## Cannot Repair into Part Components
for comp in components
    if BOM[comp].comp_type == "part"
        for node in nodes
            @constraint(m, r[node, comp] == 0)
        end
    end
end

## Cannot Complete Repairs of Difficulty at Nodes with Insufficient Repair Capability
for node in nodes, comp in components
    if BOM[comp].repair_difficulty > node_dict[node].repair_capability
        @constraint(m, r[node, comp] == 0)
    end
end

# Objective
@objective(m, Min,
    sum(arc_dict[arc].trans_cost * x[arc, comp]
        for comp in components, arc in arcs) + # transport cost
    sum(get(mult_dict, (node, comp), Mults("", "", 1.)).repair_cost_mult *
        BOM[comp].repair_cost * r[node, comp] # repair cost multipliers
        for comp in components, node in nodes) # repair cost
    )

# Solve
set_silent(m)
optimize!(m)

```

Set parameter Username

Set parameter LicenseID to value 2807856

Academic license - for non-commercial use only - expires 2027-04-15

Results: Level 1

```

In [10]: # Output
println("Optimal Objective Value: ", round(objective_value(m), digits=2)) # print objective value
println()

## Flow Output
flow_output_df = DataFrame(comp_id = String[],
                           node_from = String[],
                           node_to = String[],
                           qty = Int[]
                           )
for comp in components, (node_from, node_to) in arcs # store optimal flow in dataframe
    val = value(x[(node_from, node_to), comp])
    if val > 1e-3
        push!(flow_output_df, (comp_id = comp,
                                node_from = node_from,
                                node_to = node_to,
                                qty = val)
        )
    end
end
sort!(flow_output_df, [:comp_id, :node_from, :node_to])
println("Optimal Flow Schedule:") # output optimal flow dataframe
println(flow_output_df)
println()

## Repair Output
repair_output_df = DataFrame(comp_id = String[],
                             node_id = String[],
                             qty = Int[]
                             )
for comp in components, node in nodes # store optimal repair in dataframe
    val = value(r[node, comp])
    if val > 1e-3
        push!(repair_output_df, (comp_id = comp,
                                   node_id = node,
                                   qty = val)
        )
    end
end

```

```
    end
end
sort!(repair_output_df, [:comp_id, :node_id,])
println("Optimal Repair Schedule:") # output optimal repairs
println(repair_output_df)

## Convert Output to CSV
mkpath("level1output")
CSV.write("level1output/flow_output.csv", flow_output_df)
CSV.write("level1output/repair_output.csv", repair_output_df);
```


Optimal Objective Value: 1434.3

Optimal Flow Schedule:

15x4 DataFrame

Row	comp_id	node_from	node_to	qty
	String	String	String	Int64
1	LRU_2	DEPOT_1	REGWH_1	2
2	LRU_2	DEPOT_1	REGWH_2	4
3	LRU_2	LOCAL_2	CUST_2	2
4	LRU_2	LOCAL_3	CUST_3	3
5	LRU_2	LOCAL_4	CUST_4	1
6	LRU_2	REGWH_1	LOCAL_2	2
7	LRU_2	REGWH_2	LOCAL_3	3
8	LRU_2	REGWH_2	LOCAL_4	1
9	LRU_2_1	SPEC_4	DEPOT_1	1
10	LRU_3	DEPOT_1	REGWH_2	2
11	LRU_3	LOCAL_3	CUST_3	2
12	LRU_3	REGWH_2	LOCAL_3	2
13	LRU_5	DEPOT_1	REGWH_1	3
14	LRU_5	LOCAL_1	CUST_1	3
15	LRU_5	REGWH_1	LOCAL_1	3

Optimal Repair Schedule:

3x3 DataFrame

Row	comp_id	node_id	qty
	String	String	Int64
1	LRU_2	DEPOT_1	6
2	LRU_3	DEPOT_1	1
3	LRU_5	DEPOT_1	3

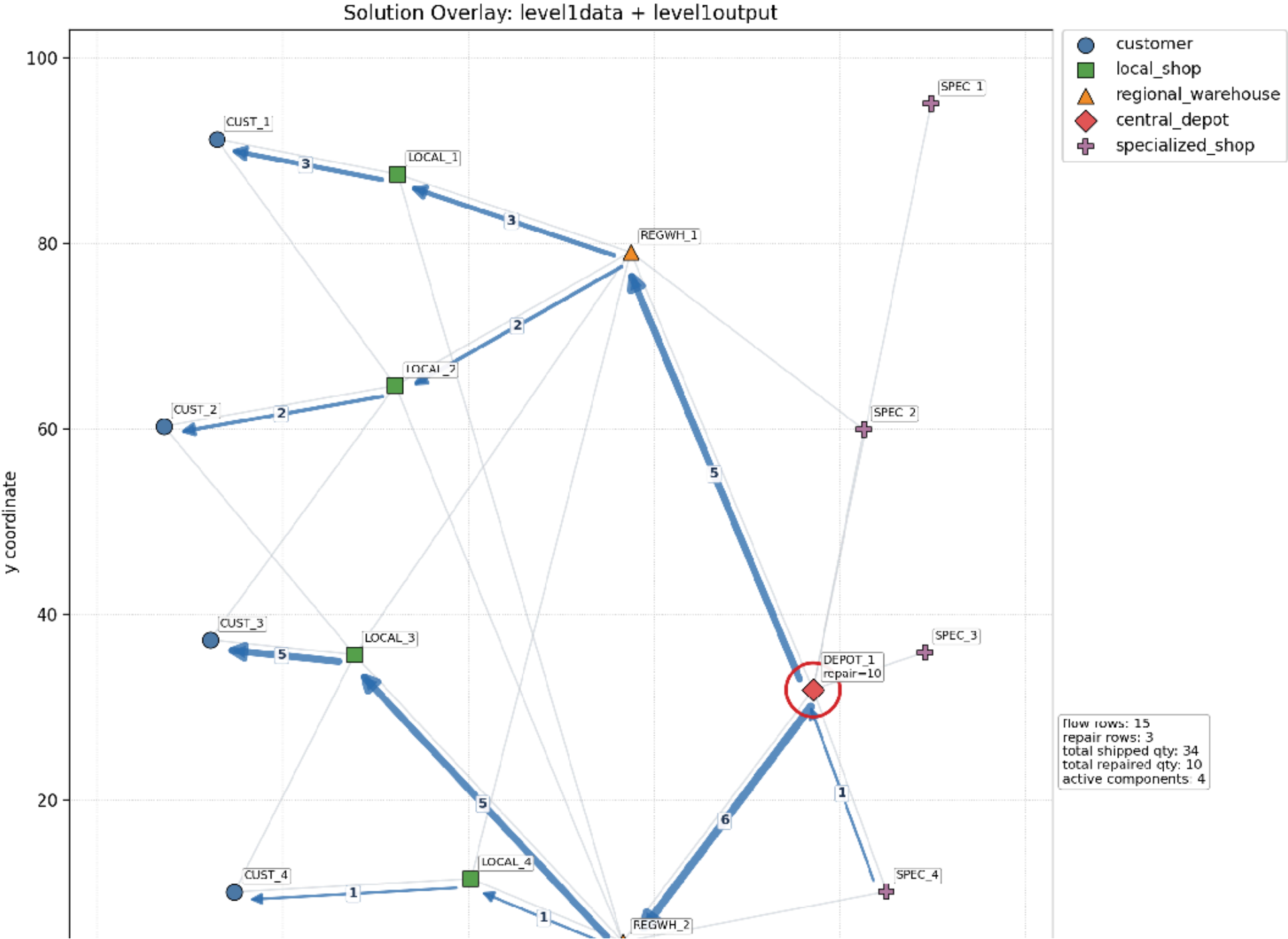
Discussion: Level 1

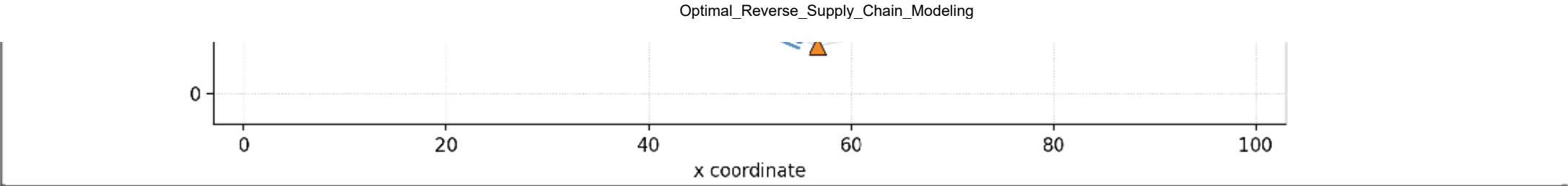
As shown in the figure below, level 1's solution represents an extreme, where all the repairs are performed at the central depot. This is expected behavior, as the depot is the cheapest of the repair options outweighing the cost of transporting components from the specialized shops, and its high skill level allows it to complete the vast majority of repairs. However, this solution breaks down under real-world conditions, as the depot would become backlogged (this limitation is addressed in level 2) and slow down repairs. Another limitation of this model is the use of shop-type-level cost multipliers. In reality, individual shops would likely have slightly different repair costs, even if they belong to the

same shop type. To simplify data generation, we assign a single shared cost multiplier to each shop type. However, the Julia model supports variable per-node repair costs, making this a straightforward extension if the user required it.

```
In [11]: plot(load("./level1output/solution_network_all.png"), axis=nothing, size=(1200, 900))
```

Out[11]:





Level 2: Math Model, Implementation, Results, and Discussion

Level 2 improves on Level 1 by capturing the notion of time in our model; in particular, the Level 2 model adds per-component repair times, repair time multipliers by location, and transportation times. In doing so, we introduce time into the objective function with a weight parameter, enabling us to study the tradeoff between cost and time in fulfilling demands. Additionally, we further refine our notion of shop abilities by introducing available shop-hours to each node, which are then consumed when completing repairs.

We mirror the existing cost logic when calculating time cost. For example, similar to repair cost, LRUs require less time to repair than SRUs, reflecting their more modular assembly process. We also use the same simplifying assumption for arc traversal: each part, SRU, and LRU incurs transportation time separately when moved through the network. For component repairs, per-node time multipliers (see `repair_time_multiplier` in `component_multiplier.csv`) are applied to the objective function. We assign regional warehouses a roughly standard repair-time multiplier of 1.1, reflecting that their primary role is storage rather than repair. Local shops are modeled as slightly faster, with a multiplier of 0.95, because their smaller size and lower repair volume may allow them to respond more quickly. Central depots are modeled as much slower, with a multiplier of 3.0, reflecting their larger repair demand and the possibility of a waitlist before repairs can begin. Specialized shops are modeled as faster, with a multiplier of 0.8, reflecting their highly trained technicians and smaller, more focused repair workload. This creates a tradeoff between cost and time, as specialized shops are substantially faster but also significantly more expensive.

We include time in the objective function by adding a time-cost term weighted by a parameter λ . This parameter controls the relative importance assigned to minimizing fulfillment time as compared to minimizing total cost. When λ is large, meaning $\lambda \gg 1$, the model prioritizes minimizing the time required to fulfill demand because changes in time dominate changes in cost. Conversely, when λ is small, meaning $\lambda \ll 1$, the model prioritizes minimizing cost because changes in cost dominate changes in time. By solving the model across a range of values for λ , we analyze the tradeoff between total cost and fulfillment time and construct a Pareto frontier: the set of non-dominated solutions plotted according to their cost and time outcomes. This frontier provides a tool for identifying the solution that best aligns with a given set of operational priorities.

In addition to adding the notion of time-cost, we limit each node's total repair time (i.e. work-hours available at the shop) to simulate each shop's finite labor supply. For simplicity, we model the shop's available hours as a single pooled value, relying on each location to assign the hours to workers. To perform repairs at some node, we now require the total repair time of conducting all repairs, including the multiplier, does not exceed the node's total available shop time (see `repair_capacity` in `nodes.csv`). To reflect their large potential throughput, depots are given the largest repair capacity, followed by warehouses and specialized shops, and finally local shops with the smallest capacity. As a result, in high-demand scenarios, the model will be forced to distributed repairs more broadly throughout the network rather than utilizing solely the central depot. This also

creates a natural prioritization effect: lower-level repairs are more likely to be assigned to lower-skilled shops, while higher-skilled shops become saturated with higher-difficulty repairs that cannot be performed elsewhere.

Math Model: Level 2

We want to modify our Level 1 math model to include the extra logic of Level 2. Specifically, we want to modify the objective function to include the transport time and repair time, with multipliers, as a tradeoff with a weight parameter λ^{time} ; additionally, we want to provide an upper bound on repairs at each location (e.g. shop-hours per day).

Additional Problem Data

Let T_k^{repair} denote the standard time it takes to repair component $k \in K$ and let T_a^{flow} denote the standard time it takes to traverse arc $a \in A$; let $m_n^{\text{time}} \geq 0$ denote the repair time multiplier at node $n \in N$; let $T_n^{\text{cap}} \geq 0$ denote the node's $n \in N$ time shop capacity.

Additional Decision Variables

N/A

Additional Constraints

Repairs at a node cannot exceed that nodes work-hour shop capacity,

$$\sum_{k \in K} m_n^{\text{time}} T_k^{\text{repair}} r_{n,k} \leq T_n^{\text{cap}}. \quad n \in N$$

Modified Objective Function

We want to minimize the time of filling demands; thus, we want to minimize the time of traversal, which is the sum over all arcs of all components being transported across an arc multiplied by the cost of traversing that arc,

$$\sum_{a \in A, k \in K} T_a^{\text{flow}} x_{a,k},$$

and we want to minimize the cost of repair, which is the sum of all repairs completed at some node multiplied by the cost of the components repair, for every node,

$$\sum_{n \in N, k \in K} m_n^{\text{time}} T_k^{\text{repair}} r_{n,k},$$

together, which gives us our time objective function as their sum,

$$\sum_{a \in A, k \in K} T_a^{flow} x_{a,k} + \sum_{n \in N, k \in K} m_n^{time} T_k^{repair} r_{n,k};$$

hence, with the trade-off weight parameter λ^{time} , we have the objective function we want to minimize as,

$$\left[\left(\sum_{a \in A, k \in K} C_a^{flow} x_{a,k} + \sum_{n \in N, k \in K} m_n^{cost} C_k^{repair} r_{n,k} \right) + \lambda^{time} \left(\sum_{a \in A, k \in K} T_a^{flow} x_{a,k} + \sum_{n \in N, k \in K} m_n^{time} T_k^{repair} r_{n,k} \right) \right].$$

Putting this together, we have the integer linear program,

$$\begin{aligned} \min \quad & \left[\left(\sum_{a \in A, k \in K} C_a^{flow} x_{a,k} + \sum_{n \in N, k \in K} m_n^{cost} C_k^{repair} r_{n,k} \right) + \right. \\ & \left. \lambda^{time} \left(\sum_{a \in A, k \in K} T_a^{flow} x_{a,k} + \sum_{n \in N, k \in K} m_n^{time} T_k^{repair} r_{n,k} \right) \right] \\ \text{s.t.} \quad & \left[I_{n,k} + \sum_{(i,n) \in A} x_{(i,n),k} + r_{n,k} \geq \right. \\ & \left. \sum_{(n,i) \in A} x_{(n,i),k} + \sum_{k' \in K_k} r_{n,k'} q_{k',k} + d_{n,k} \right], & n \in N, k \in K \\ & r_{n,k} = 0, & n \in N \setminus N_k, k \in K \\ & \sum_{k \in K} m_n^{time} T_k^{repair} r_{n,k} \leq T_n^{cap}, & n \in N \\ & x_{a,k} \geq 0, & a \in A, k \in K \\ & r_{n,k} \geq 0, & n \in N, k \in K \end{aligned}$$

Implementation: Level 2

We first load in level 2's data.

```
In [12]: # Load Data
node_df = CSV.read("level2data/nodes.csv", DataFrame,delim=',')
arc_df = CSV.read("level2data/arcs.csv", DataFrame,delim=',')
bom_df = CSV.read("level2data/bom.csv", DataFrame,delim=',')
demand_df = CSV.read("level2data/demand.csv", DataFrame,delim=',')
node_inv_df = CSV.read("level2data/node_inventory.csv", DataFrame,delim=',')
mult_df = CSV.read("level2data/component_multiplier.csv", DataFrame,delim=',')

# Convert node_df to Node Information
#-----
struct Node # Node struct
  name::String
  node_type::String
  loc::Tuple{Float64, Float64}
  repair_capability::Int
  repair_capacity::Int
end
node_dict = Dict() # dictionary of nodes -> Node struct
for df_node in eachrow(node_df) # add nodes to node_dict
  node_id = df_node.node_id
  node_type = df_node.node_type
  (x, y) = (df_node.x, df_node.y)
  repair_capability = df_node.repair_capability
  repair_capacity = df_node.repair_capacity
  node = Node(node_id, node_type, (x, y), repair_capability, repair_capacity)
  node_dict[node_id] = node
end
nodes = collect(keys(node_dict)) # list of nodes

# Convert arc_df to Arc Information
#-----
struct Arc # Arc struct
  name::String
  fromto::Tuple{String, String}
  trans_cost::Float64
```

```

    trans_time::Float64
end
arc_dict = Dict() # dictionary of arcs -> Arc struct
for df_arc in eachrow(arc_df) # add arcs to arc_dict
    arc_id = df_arc.arc_id
    node_from = df_arc.from_node
    node_to = df_arc.to_node
    trans_cost = df_arc.cost
    trans_time = df_arc.time_cost
    arc = Arc(arc_id, (node_from, node_to), trans_cost, trans_time)
    arc_dict[(node_from, node_to)] = arc
end
arcs = collect(keys(arc_dict)) # list of arcs of network
incoming = Dict() # dictionary of node -> nodes incident to the node
outgoing = Dict() # dictionary of node -> nodes incident from the node
for (node_from, node_to) in arcs # add nodes to incoming and outgoing
    push!(get!(incoming, node_to, []), node_from)
    push!(get!(outgoing, node_from, []), node_to)
end

# Convert bom_df to Bill of Material (BOM) Information
#-----
struct Component # Component struct
    comp_id::String
    comp_type::String
    repair_difficulty::Int
    repair_cost::Float64
    repair_time::Float64
    subcomps::Vector{Tuple{String, Int}}
end
BOM = Dict() # dictionary of component -> Component struct
for bom_item in eachrow(bom_df) # build dictionary
    comp_id = bom_item.parent_component_id
    comp_type = bom_item.parent_component_class
    comp_repair_difficulty = bom_item.parent_component_repair_difficulty
    comp_repair_cost = bom_item.parent_component_cost
    comp_repair_time = bom_item.parent_component_repair_time
    comp_repair_list = []
    sub_bom = subset(bom_df, :parent_component_id => ByRow(==(comp_id)))
    for sub_comp in eachrow(sub_bom)

```



```

        sub_comp_id = sub_comp.child_component_id
        sub_comp_qty = sub_comp.quantity_required
        push!(comp_repair_list, (sub_comp_id, sub_comp_qty))
    end
    comp = Component(comp_id, comp_type, comp_repair_difficulty,
                     comp_repair_cost, comp_repair_time, comp_repair_list
                     )
    BOM[comp_id] = comp
end
part_bom = subset(bom_df, :child_component_class => ByRow(=="part")) # add parts components to BOM
for bom_item in eachrow(part_bom)
    part_id = bom_item.child_component_id
    part_type = "part"
    part_repair_difficulty = 0
    part_repair_cost = 0
    part_repair_time = 0
    part_repair_list = []
    part = Component(part_id, part_type, part_repair_difficulty,
                     part_repair_cost, part_repair_time, part_repair_list
                     )
    BOM[part_id] = part
end
components = collect(keys(BOM)) # List of components in BOM
subcomp_dict = Dict() # dictionary of comonents -> List of subcomponents
for comp in components
    subcomp_bom = subset(bom_df, :child_component_id => ByRow(==(comp)))
    subcomp_for_lst = []
    for subcomp_for in eachrow(subcomp_bom)
        push!(subcomp_for_lst, (subcomp_for.parent_component_id, subcomp_for.quantity_required))
    end
    subcomp_dict[comp] = subcomp_for_lst
end

# Convert demand_df to Demand Information
#-----
struct Demand # Demand struct
    name::String
    loc::String
    component::String
    qty::Int

```

```

end
demands = Dict() # dictionary of each node and component pair -> Demand struct
for df_demand in eachrow(demand_df) # build dictionary
    demand_id = df_demand.demand_id
    demand_node = df_demand.node_id
    demand_comp = df_demand.component_id
    qty = df_demand.quantity
    demand = Demand(demand_id, demand_node, demand_comp, qty)
    demands[(demand_node, demand_comp)] = demand
end

# Convert node_inv_df to Node Inventory Information
#-----
struct Stock # Stock struct
    loc::String
    component::String
    qty::Int
end
inventory = Dict() # dictionary of each node and component pair -> Stock struct
for df_stock in eachrow(node_inv_df) # build dictionary
    stock_node = df_stock.node_id
    stock_comp = df_stock.component_id
    qty = df_stock.quantity
    stock = Stock(stock_node, stock_comp, qty)
    inventory[(stock_node, stock_comp)] = stock
end

# Convert comp_mult_df to Component Multiplier Information
#-----
struct Mults # Mult struct
    loc::String
    component::String
    repair_cost_mult::Float64
    repair_time_mult::Float64
end
mult_dict = Dict() # dictionary of each Location and component -> Comp_Mult struct
for df_mults in eachrow(mult_df) # build dictionary
    mults_node = df_mults.node_id

```

```

mults_comp = df_mults.component_id
repair_cost_mult = df_mults.repair_cost_multiplier # change column name to repair_cost_mult
repair_time_mult = df_mults.repair_time_multiplier # change column name to repair_time_mult
mults = Mults(mults_node, mults_comp, repair_cost_mult, repair_time_mult)
mult_dict[(mults_node, mults_comp)] = mults
end

```

We then implement the discussed model in Julia.

```

In [13]: # Parameter
lambda_time = 1 # lambda parameter

# Model
m = Model(Gurobi.Optimizer)

# Variables
@variable(m, x[arc in arcs, comp in components] >= 0, Int) # flow of x component through arc
@variable(m, r[node in nodes, comp in components] >= 0, Int) # r repairs of component at node

# Constraints
## Flow Conservation
for comp in components, node in nodes
    inflow = sum(x[(node_from, node), comp]
                 for node_from in incoming[node]) # flow of component into node
    outflow = sum(x[(node, node_to), comp]
                 for node_to in outgoing[node]) # flow of component out of node
    demand = get(demands, (node, comp), Demand("", "", "", 0)).qty # demand of component present at node
    stock = get(inventory, (node, comp), Stock("", "", 0)).qty # initial stock of component at node
    repairs = r[node, comp] # repairs of component at node
    consumed = sum(qty * r[node, comp_repaired]
                  for (comp_repaired, qty) in subcomp_dict[comp]; init=0) # component consumed for repairs at node
    @constraint(m, stock + inflow + repairs >= outflow + consumed + demand)
end

```

```

## Cannot Repair into Part Components
for comp in components
    if BOM[comp].comp_type == "part"
        for node in nodes
            @constraint(m, r[node, comp] == 0)
        end
    end
end

## Limit Node Repairs to Repair Capacity Hours
for node in nodes
    @constraint(m, sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
        * BOM[comp].repair_time * r[node, comp] for comp in components)
        <= node_dict[node].repair_capacity)
end

## Cannot Complete Repairs of Difficulty at Nodes with Insufficient Repair Capability
for node in nodes, comp in components
    if BOM[comp].repair_difficulty > node_dict[node].repair_capability
        @constraint(m, r[node, comp] == 0)
    end
end

# Objective
@objective(m, Min,
    sum(arc_dict[arc].trans_cost * x[arc, comp]
    for comp in components, arc in arcs) + # transport cost
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_cost_mult
        * BOM[comp].repair_cost * r[node, comp]
    for comp in components, node in nodes) + # repair cost
    lambda_time * (sum(arc_dict[arc].trans_time * x[arc, comp]
    for comp in components, arc in arcs) + # transport time
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
        * BOM[comp].repair_time * r[node, comp]
    for comp in components, node in nodes)) # repair time

```

```
)

# Solve
set_silent(m)
optimize!(m)
```

Set parameter Username

Set parameter LicenseID to value 2807856

Academic license - for non-commercial use only - expires 2027-04-15

Results: Level 2

```
In [14]: # Output
println("Optimal Objective Value: ", round(objective_value(m), digits=2)) # print objective value
println()

## Flow Output
flow_output_df = DataFrame(comp_id = String[],
                           node_from = String[],
                           node_to = String[],
                           qty = Int[]
                           )
for comp in components, (node_from, node_to) in arcs # store optimal flow in dataframe
    val = value(x[(node_from, node_to), comp])
    if val > 1e-3
        push!(flow_output_df, (comp_id = comp,
                               node_from = node_from,
                               node_to = node_to,
                               qty = val)
        )
    end
end
sort!(flow_output_df, [:comp_id, :node_from, :node_to])
println("Optimal Flow Schedule:") # output optimal flow dataframe
println(flow_output_df)
println()
```

```
## Repair Output
repair_output_df = DataFrame(comp_id = String[],
                             node_id = String[],
                             qty = Int[]
                             )
for comp in components, node in nodes # store optimal repair in dataframe
  val = value(r[node, comp])
  if val > 1e-3
    push!(repair_output_df, (comp_id = comp,
                             node_id = node,
                             qty = val)
    )
  end
end
sort!(repair_output_df, [:comp_id, :node_id,])
println("Optimal Repair Schedule:") # output optimal repairs
println(repair_output_df)

## Convert Output to CSV
mkpath("level2output")
CSV.write("level2output/flow_output.csv", flow_output_df)
CSV.write("level2output/repair_output.csv", repair_output_df);
```

Optimal Objective Value: 4302.5

Optimal Flow Schedule:

23x4 DataFrame

Row	comp_id	node_from	node_to	qty
	String	String	String	Int64
1	LRU_2	DEPOT_1	REGWH_2	3
2	LRU_2	LOCAL_1	CUST_2	2
3	LRU_2	LOCAL_3	CUST_3	3
4	LRU_2	LOCAL_4	CUST_4	1
5	LRU_2	REGWH_1	LOCAL_1	2
6	LRU_2	REGWH_2	LOCAL_3	3
7	LRU_2	REGWH_2	LOCAL_4	1
8	LRU_2	SPEC_1	REGWH_1	2
9	LRU_2	SPEC_2	DEPOT_1	2
10	LRU_2	SPEC_4	REGWH_2	1
11	LRU_2_2	DEPOT_1	SPEC_2	2
12	LRU_2_2	REGWH_1	SPEC_1	2
13	LRU_2_2	REGWH_2	SPEC_4	1
14	LRU_3	DEPOT_1	REGWH_2	2
15	LRU_3	LOCAL_3	CUST_3	2
16	LRU_3	REGWH_2	LOCAL_3	2
17	LRU_3	SPEC_3	DEPOT_1	1
18	LRU_3_1	DEPOT_1	SPEC_3	1
19	LRU_5	DEPOT_1	REGWH_1	3
20	LRU_5	LOCAL_1	CUST_1	3
21	LRU_5	REGWH_1	LOCAL_1	3
22	LRU_5	SPEC_3	DEPOT_1	2
23	LRU_5_1	DEPOT_1	SPEC_3	4

Optimal Repair Schedule:

7x3 DataFrame

Row	comp_id	node_id	qty
	String	String	Int64
1	LRU_2	DEPOT_1	1
2	LRU_2	SPEC_1	2
3	LRU_2	SPEC_2	2
4	LRU_2	SPEC_4	1
5	LRU_3	SPEC_3	1

6		LRU_5	DEPOT_1	1
7		LRU_5	SPEC_3	2

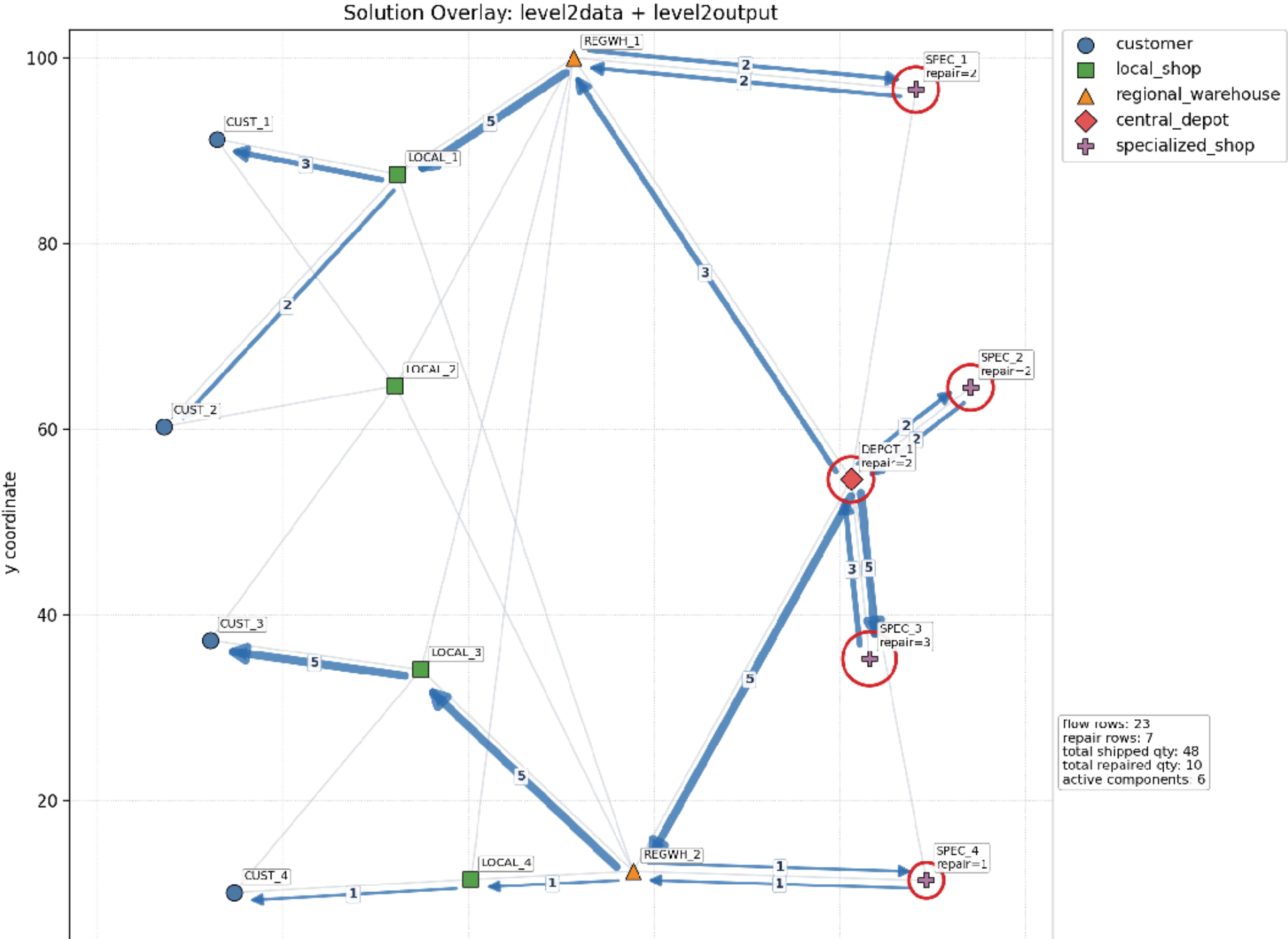
Discussion: Level 2

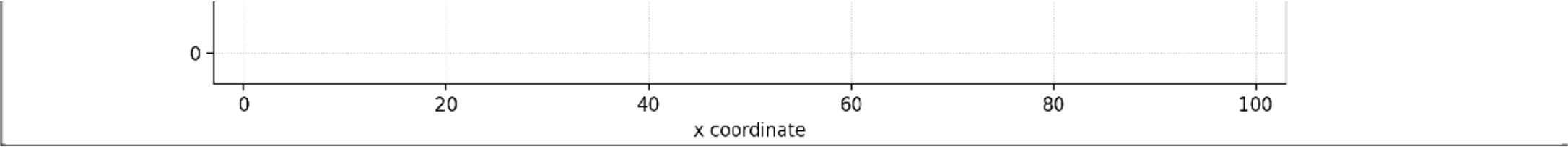
We first examine the solution under an objective that assigns equal weight to time and cost. We then perform a Pareto curve analysis to illustrate the tradeoffs between these two objectives.

As shown in the figure below, incorporating time into the objective introduces more nuanced and realistic behavior. For example, some components are routed to specialized shops, which are more expensive but offer faster repair times, before being shipped back to central depot or regional warehouses and ultimately delivered to the customer. In contrast, other repairs are performed locally at central depot to reduce cost, even if this results in longer repair times.

```
In [15]: plot(load("./level2output/solution_network_all.png"), axis=nothing, size=(1200, 900))
```


Out[15]:

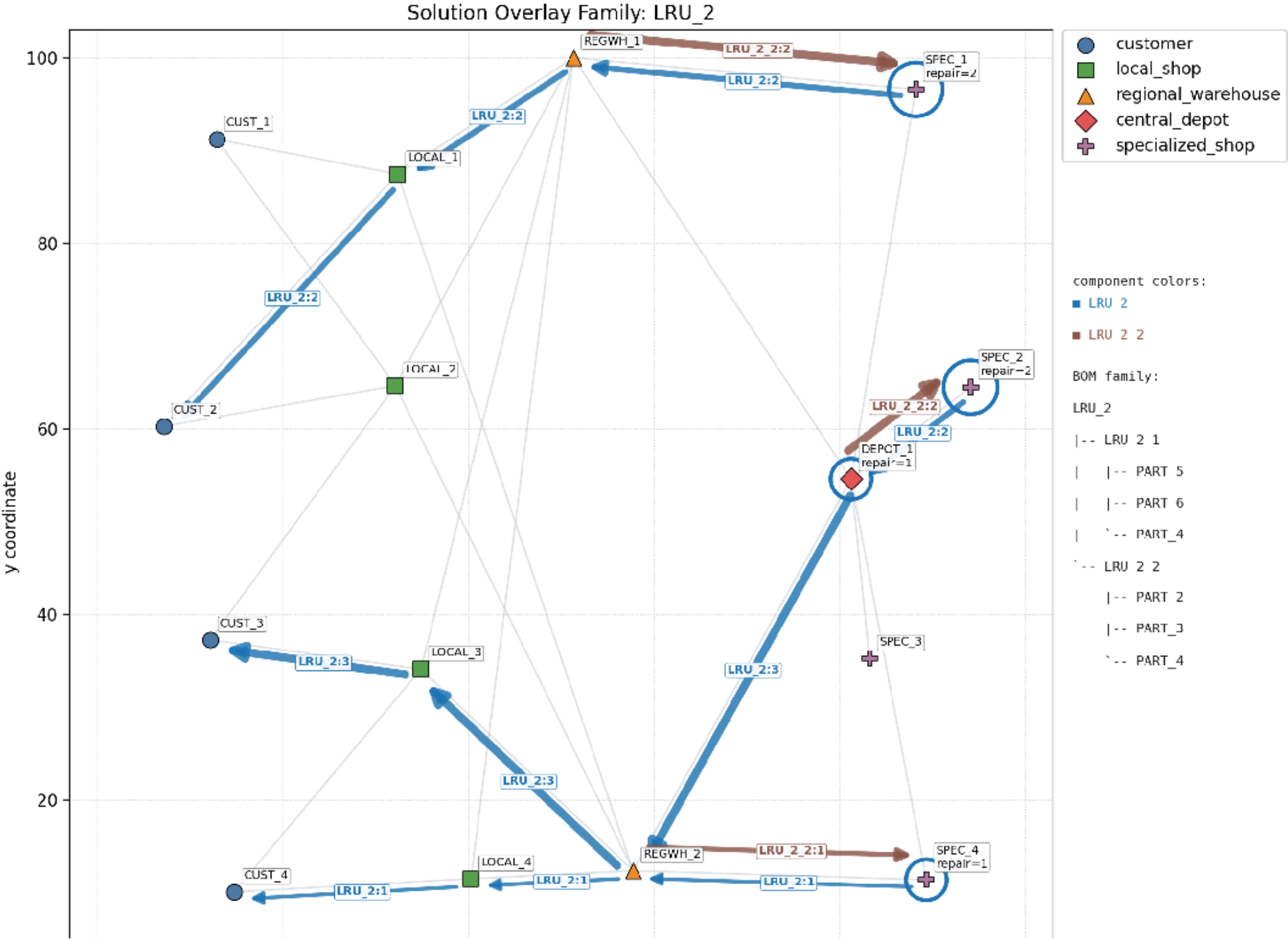


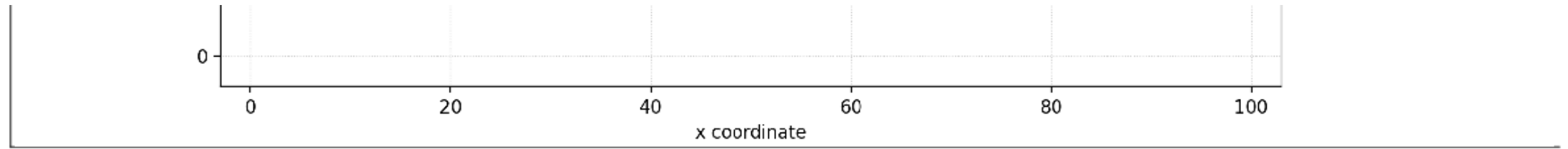


The behavior is even more interesting if we examine a specific component. As shown in the figure below, one of LRU_2's SRUs (shown as LRU_2_2) is shipped from DEPOT_1 to SPEC_2, where a LRU_2 unit is repaired and then shipped back to the depot. At the same time, there are repairs of a separate unit of LRU_2 occurring at the central depot. This shows that the model made the choice to pay extra shipping cost/time for one unit of LRU_2, while in the other case, it chose to repair a unit of LRU_2 in place.

```
In [16]: plot(load("./level2output/solution_family_lru_2.png"), axis=nothing, size=(1200, 900))
```

Out[16]:





One additional observation not captured in the final results is that the assumptions encoded in the data generation process strongly pulled the optimal solution away from our intentions. Initially, we observed that nearly all repairs were still performed at the central depot, which was unexpected given its longer repair times. Further analysis revealed two main causes. First, the capacities of smaller shops were too limited, preventing them from absorbing much of the repair workload. Second, the depot's repair-cost discount outweighed the disadvantage of its slower repair time. To address this, we tuned the data generation process by increasing the repair capacities of smaller shops and increasing the time multiplier assigned to the central depot.

We found this result instructive because, even with an effective model formulation, the structure of the input data created a potentially unrealistic solution. This highlights an important practical lesson: in real-world optimization settings, data design and calibration can be just as important as the mathematical model itself. Of course, in a practical implementation, the data itself would be generated from statistical averages and historical trends which does much of the calibration automatically.

Below we explore various tradeoffs between cost and time and visualize the results as a Pareto curve.

```
In [17]: lambdas = sort([0, 0.5, 1., 1.5, 2., 2.5, 3., 3.5, 4., 4.5, 5., 10., 20., 25, 50]) # tradeoff parameter
opt_costs = []
opt_times = []

for lambda_time in lambdas
    # Model
    m = Model(Gurobi.Optimizer)

    # Variables
    @variable(m, x[arc in arcs, comp in components] >= 0, Int) # flow of x component through arc
    @variable(m, r[node in nodes, comp in components] >= 0, Int) # r repairs of component at node

    # Constraints
```

```

## Flow Conservation
for comp in components, node in nodes
    inflow = sum(x[(node_from, node), comp]
                 for node_from in incoming[node])           # flow of component into node
    outflow = sum(x[(node, node_to), comp]
                 for node_to in outgoing[node])             # flow of component out of node
    demand = get(demands, (node, comp), Demand("", "", "", 0)).qty # demand of component present at node
    stock = get(inventory, (node, comp), Stock("", "", 0)).qty # initial stock of component at node
    repairs = r[node, comp] # repairs of component at node
    consumed = sum(qty * r[node, comp_repaired]
                  for (comp_repaired, qty) in subcomp_dict[comp]; init=0) # component consumed for repairs at node
    @constraint(m, stock + inflow + repairs >= outflow + consumed + demand)
end

## Cannot Repair into Part Components
for comp in components
    if BOM[comp].comp_type == "part"
        for node in nodes
            @constraint(m, r[node, comp] == 0)
        end
    end
end

## Limit Node Repairs to Repair Capacity Hours
for node in nodes
    @constraint(m, sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
                      * BOM[comp].repair_time * r[node, comp] for comp in components)
               <= node_dict[node].repair_capacity)
end

## Cannot Complete Repairs of Difficulty at Nodes with Insufficient Repair Capability
for node in nodes, comp in components
    if BOM[comp].repair_difficulty > node_dict[node].repair_capability
        @constraint(m, r[node, comp] == 0)
    end
end

```

```

# Objective
@objective(m, Min,
    sum(arc_dict[arc].trans_cost * x[arc, comp]
        for comp in components, arc in arcs) + # transport cost
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_cost_mult
        * BOM[comp].repair_cost * r[node, comp]
        for comp in components, node in nodes) + # repair cost
    lambda_time * (sum(arc_dict[arc].trans_time * x[arc, comp]
        for comp in components, arc in arcs) + # transport time
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
        * BOM[comp].repair_time * r[node, comp]
        for comp in components, node in nodes)) # repair time
)

# Solve
set_silent(m)
optimize!(m)

# Extract Values
opt_cost = sum(arc_dict[arc].trans_cost * value(x[arc, comp])
    for comp in components, arc in arcs) + # transport cost
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_cost_mult
        * BOM[comp].repair_cost * value(r[node, comp])
        for comp in components, node in nodes) # repair cost
opt_time = sum(arc_dict[arc].trans_time * value(x[arc, comp])
    for comp in components, arc in arcs) + # transport time
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
        * BOM[comp].repair_time * value(r[node, comp])
        for comp in components, node in nodes) # repair time
push!(opt_costs, opt_cost)
push!(opt_times, opt_time)
end

```

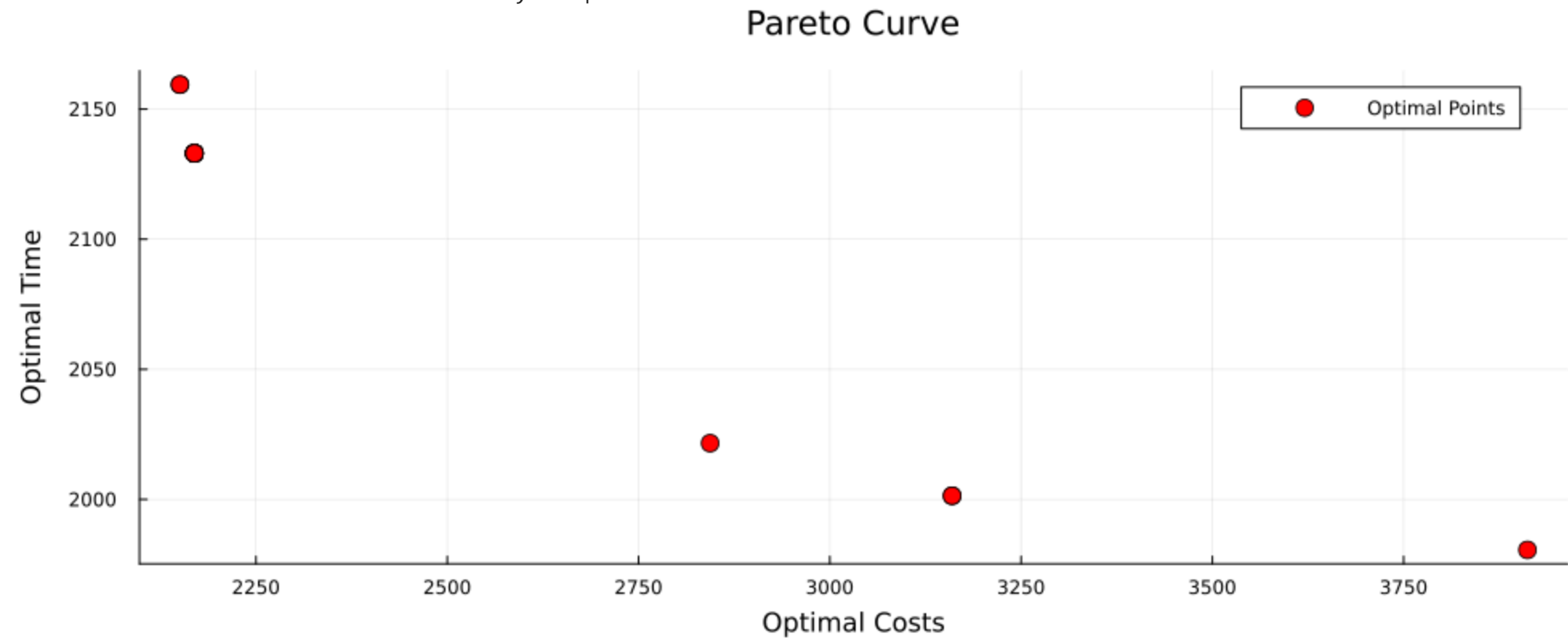
```
# Plot Pareto Curve
p = scatter(opt_costs, opt_times, marker=:circle, markersize=6,
            label="Optimal Points", color=:red)
title!(p, "Pareto Curve")
xlabel!(p, "Optimal Costs")
ylabel!(p, "Optimal Time")

# Display
plot(p, size=(1000, 400), margin=5Plots.mm)
```

[illegible]


```
Set parameter Username
Set parameter LicenseID to value 2807856
Academic license - for non-commercial use only - expires 2027-04-15
```

Out[17]:



As shown in the figure above, multiple optimal solutions emerge depending on the weight placed on time. In the extreme case, when the time-cost weight is set to 50, the vast majority of repairs occur at specialized shops because of their lower turnaround times, despite their higher repair costs. This illustrates how a company could use our model to evaluate alternative repair strategies based on its desired fulfillment speed and willingness to incur additional cost.

Level 3: Math Model, Implementation, Results, and Discussion

Level 3 improves on Level 2 by expanding our model to include multi-day planning and back orders. To do so, we modify demand to occur over multiple time-slices. Once a day's demand is satisfied, any remaining node inventory carries forward to the next day, with each independent day behaving similarly to the Level 2 model. Additionally, we modify our model to allow demand to be backordered across days at a high penalty cost. For the purpose of our objective function, we sum each time slice's respective values with the given tradeoff weight parameter, ensuring we are optimizing the entire multi-day demand scenario rather than a single time-slice.

To implement multi-day planning, we introduce a daily stock variable for each component at each node, including the initial system state. This stock variable tracks how many units of a component remain at a node at the end of each time slice. Any leftover inventory then carries forward as the starting inventory for the next time slice. As before, each node-component pair has a net gain and net loss term. However, unlike the previous formulation, the net gain now includes backorders for that component, while the net loss includes leftover stock carried forward from the previous time slice. Because we now track stock both before and after each time slice, we can replace the previous flow-balance inequality with an equality. Net gain must equal net loss, as it is physically impossible to have negative components or generate components from nothing. Additionally, we enforce conditions like shop-time limits, network flow, and non-negativity conditions on a per time-slice basis, with shop-time limits resetting between time periods.

Backorders provide a way to temporarily defer demand at a significant cost, while still requiring that the demand be satisfied in a later time slice. This expands the set of feasible demand scenarios: demand that would otherwise be infeasible within a single time slice due to shop capacity or work-time limits can instead be backordered and fulfilled in a future time slice. We assume that the backorder penalty is a flat cost with no additional time component. Therefore, we model the total backorder cost as the number of backordered components across the network multiplied by a fixed penalty. We also assume that the system begins with no existing backorders and ends with no unresolved backorders. In other words, backorders are fixed to zero at time slice 0 and at the final time slice.

Math Model: Level 3

We want to modify our Level 2 math model to include the extra logic of Level 3. Specifically, we want to take Level 2 and turn it into a multi-period planning problem; additionally, we want to allow for backlogging demands at an elevated cost in case repairs cannot be met on a given timestep.

Additional Problem Data

Let $\mathcal{T}$ denote the set of timesteps (e.g. days).

Additional Decision Variables and Decision Variable Modifications

Let $s_{n,k,t} \in \mathbb{Z}$ denote the number of component $k \in K$ stored at node $n \in N$ at time $t \in \mathcal{T} \cup \{0\}$ where $t = 0$ denotes the systems initial state; let $b_{n,k,t} \in \mathbb{Z}$ denote the number of component $k \in K$ backordered at node $n \in N$ at time $t \in \mathcal{T} \cup \{0\}$. In addition, we append a time index $t \in \mathcal{T}$ for all transport and repair decision variables, $x_{a,k,t}$ and $r_{n,k,t}$. Let B denote a flat penalty cost to any backordering.

Additional Constraints

We cannot allow for negative stock nor negative backorders at backorders; thus, we have the nonnegativity constraints,

$$\begin{aligned} s_{n,k,t} &\geq 0, & n \in N, k \in K, t \in \mathcal{T} \\ b_{n,k,t} &\geq 0. & n \in N, k \in K, t \in \mathcal{T} \end{aligned}$$

The initial stock at each node is the inventory,

$$s_{n,k,0} = I_{n,k}. \quad n \in N, k \in K$$

We do not start with any backorders and we must fulfill all backorders by the end of the timesteps $t^{\text{end}} \in \mathcal{T}$,

$$b_{n,k,t} = 0. \quad t \in \{0, t^{\text{end}}\}$$

We must modify our flow constraint so that it applies over each timestep $t \in \mathcal{T}$ as well as properly balance the constraint using the stock over time timestep,

$$\left[s_{n,k,t-1} + \sum_{(i,n) \in A} x_{(i,n),k,t} + r_{n,k,t} + b_{n,k,t} = s_{n,k,t} + \sum_{(n,i) \in A} x_{(n,i),k,t} + \sum_{k' \in K_k} r_{n,k',t} q_{k',k} + d_{n,k} \right], \quad n \in N, k \in K, t \in \mathcal{T}$$

which represents *initial stock + inflow + repairs + backorders = final stock + outflow + consumed + demand* for each node, component, and time-slice.

In addition, we append the proper time index $t \in \mathcal{T}$ for all constraints from Level 2.

Objective Function Modifications

We additionally have a flat cost for any backorders, represented as the sum of all backorders performed anywhere in the network multiplied by the flat backorder penalty B ,

$$\sum_{n \in N, k \in K, t \in \mathcal{T}} B b_{n,k,t};$$

hence, adding this backorder penalty, we have the objective function we want to minimize as,

$$\left[\left(\sum_{a \in A, k \in K, t \in \mathcal{T}} C_a^{\text{flow}} x_{a,k,t} + \sum_{n \in N, k \in K, t \in \mathcal{T}} m_n^{\text{cost}} C_k^{\text{repair}} r_{n,k,t} + \sum_{n \in N, k \in K, t \in \mathcal{T}} B b_{n,k,t} \right) + \lambda^{\text{time}} \left(\sum_{a \in A, k \in K, t \in \mathcal{T}} T_a^{\text{flow}} x_{a,k,t} + \sum_{n \in N, k \in K, t \in \mathcal{T}} m_n^{\text{time}} T_k^{\text{repair}} r_{n,k,t} \right) \right]$$

Putting this together, we have the integer linear program,

$$\begin{aligned} \min \quad & \left[\left(\sum_{a \in A, k \in K, t \in \mathcal{T}} C_a^{flow} x_{a,k,t} + \sum_{n \in N, k \in K, t \in \mathcal{T}} m_n^{\text{cost}} C_k^{repair} r_{n,k,t} + \right. \right. \\ & \left. \left. \sum_{n \in N, k \in K, t \in \mathcal{T}} B b_{n,k,t} \right) + \right. \\ & \left. \lambda^{\text{time}} \left(\sum_{a \in A, k \in K, t \in \mathcal{T}} T_a^{flow} x_{a,k,t} + \sum_{n \in N, k \in K, t \in \mathcal{T}} m_n^{\text{time}} T_k^{repair} r_{n,k,t} \right) \right] \\ \text{s.t.} \quad & \left[s_{n,k,t-1} + \sum_{(i,n) \in A} x_{(i,n),k,t} + r_{n,k,t} + b_{n,k,t} = \right. \\ & \left. s_{n,k,t} + \sum_{(n,i) \in A} x_{(n,i),k,t} + \sum_{k' \in K_k} r_{n,k',t} q_{k',k} + d_{n,k} \right], \\ & r_{n,k,t} = 0, \\ & \sum_{k \in K} m_n^{\text{time}} T_k^{repair} r_{n,k,t} \leq T_n^{\text{cap}}, \\ & x_{a,k,t} \geq 0, \\ & r_{n,k,t} \geq 0, \\ & s_{n,k,t} \geq 0, \\ & b_{n,k,t} \geq 0. \end{aligned}$$

$$\begin{aligned} & n \in N, k \in K, t \in \mathcal{T} \\ & n \in N \setminus N_k, k \in K, t \in \mathcal{T} \\ & n \in N, t \in \mathcal{T} \\ & a \in A, k \in K, t \in \mathcal{T} \\ & n \in N, k \in K, t \in \mathcal{T} \\ & n \in N, k \in K, t \in \mathcal{T} \\ & n \in N, k \in K, t \in \mathcal{T} \end{aligned}$$

Implementation: Level 3

As before, we first load in the Level 3 data.

```
In [18]: # Load Data
node_df = CSV.read("level3data/nodes.csv", DataFrame,delim=',')
arc_df = CSV.read("level3data/arcs.csv", DataFrame,delim=',')
bom_df = CSV.read("level3data/bom.csv", DataFrame,delim=',')
demand_df = CSV.read("level3data/demand.csv", DataFrame,delim=',')
node_inv_df = CSV.read("level3data/node_inventory.csv", DataFrame,delim=',')
mult_df = CSV.read("level3data/component_multiplier.csv", DataFrame,delim=',')
```

```

# Convert node_df to Node Information
#-----
struct Node # Node struct
  name::String
  node_type::String
  loc::Tuple{Float64, Float64}
  repair_capability::Int
  repair_capacity::Int
end
node_dict = Dict() # dictionary of nodes -> Node struct
for df_node in eachrow(node_df) # add nodes to node_dict
  node_id = df_node.node_id
  node_type = df_node.node_type
  (x, y) = (df_node.x, df_node.y)
  repair_capability = df_node.repair_capability
  repair_capacity = df_node.repair_capacity
  node = Node(node_id, node_type, (x, y), repair_capability, repair_capacity)
  node_dict[node_id] = node
end
nodes = collect(keys(node_dict)) # list of nodes


# Convert arc_df to Arc Information
#-----
struct Arc # Arc struct
  name::String
  fromto::Tuple{String, String}
  trans_cost::Float64
  trans_time::Float64
end
arc_dict = Dict() # dictionary of arcs -> Arc struct
for df_arc in eachrow(arc_df) # add arcs to arc_dict
  arc_id = df_arc.arc_id
  node_from = df_arc.from_node
  node_to = df_arc.to_node
  trans_cost = df_arc.cost
  trans_time = df_arc.time_cost
  arc = Arc(arc_id, (node_from, node_to), trans_cost, trans_time)
  arc_dict[(node_from, node_to)] = arc
end

```

```

arcs = collect(keys(arc_dict)) # List of arcs of network
incoming = Dict() # dictionary of node -> nodes incident to the node
outgoing = Dict() # dictionary of node -> nodes incident from the node
for (node_from, node_to) in arcs # add nodes to incoming and outgoing
    push!(get!(incoming, node_to, []), node_from)
    push!(get!(outgoing, node_from, []), node_to)
end

# Convert bom_df to Bill of Material (BOM) Information
#-----
struct Component # Component struct
    comp_id::String
    comp_type::String
    repair_difficulty::Int
    repair_cost::Float64
    repair_time::Float64
    subcomps::Vector{Tuple{String, Int}}
end
BOM = Dict() # dictionary of component -> Component struct
for bom_item in eachrow(bom_df) # build dictionary
    comp_id = bom_item.parent_component_id
    comp_type = bom_item.parent_component_class
    comp_repair_difficulty = bom_item.parent_component_repair_difficulty
    comp_repair_cost = bom_item.parent_component_cost
    comp_repair_time = bom_item.parent_component_repair_time
    comp_repair_list = []
    sub_bom = subset(bom_df, :parent_component_id => ByRow(==(comp_id)))
    for sub_comp in eachrow(sub_bom)
        sub_comp_id = sub_comp.child_component_id
        sub_comp_qty = sub_comp.quantity_required
        push!(comp_repair_list, (sub_comp_id, sub_comp_qty))
    end
    comp = Component(comp_id, comp_type, comp_repair_difficulty,
                     comp_repair_cost, comp_repair_time, comp_repair_list)
    BOM[comp_id] = comp
end
part_bom = subset(bom_df, :child_component_class => ByRow(=="part")) # add parts components to BOM
for bom_item in eachrow(part_bom)
    part_id = bom_item.child_component_id

```

```

part_type = "part"
part_repair_difficulty = 0
part_repair_cost = 0
part_repair_time = 0
part_repair_list = []
part = Component(part_id, part_type, part_repair_difficulty,
                  part_repair_cost, part_repair_time, part_repair_list
                  )
BOM[part_id] = part
end
components = collect(keys(BOM)) # List of components in BOM
subcomp_dict = Dict() # dictionary of comonents -> list of subcomponents
for comp in components
    subcomp_bom = subset(bom_df, :child_component_id => ByRow(==(comp)))
    subcomp_for_lst = []
    for subcomp_for in eachrow(subcomp_bom)
        push!(subcomp_for_lst, (subcomp_for.parent_component_id, subcomp_for.quantity_required))
    end
    subcomp_dict[comp] = subcomp_for_lst
end

# Convert demand_df to Demand Information
#-----
timesteps = 1
struct Demand # Demand struct
    name::String
    loc::String
    component::String
    total_qty::Int
    time_schedule::Vector
end
demands = Dict() # dictionary of each node and component pair -> Demand struct
for df_demand in eachrow(demand_df) # build dictionary
    demand_id = df_demand.demand_id
    demand_node = df_demand.node_id
    demand_comp = df_demand.component_id
    total_qty = df_demand.quantity
    time_schedule = []
    for slice in names(df_demand, r"^time_slice_")
        push!(time_schedule, df_demand[slice])
    end
end

```

```

    end
    timesteps = length(time_schedule)
    demand = Demand(demand_id, demand_node, demand_comp, total_qty, time_schedule)
    demands[(demand_node, demand_comp)] = demand
end

# Convert node_inv_df to Node Inventory Information
#-----
struct Stock # Stock struct
    loc::String
    component::String
    qty::Int
end
inventory = Dict() # dictionary of each node and component pair -> Stock struct
for df_stock in eachrow(node_inv_df) # build dictionary
    stock_node = df_stock.node_id
    stock_comp = df_stock.component_id
    qty = df_stock.quantity
    stock = Stock(stock_node, stock_comp, qty)
    inventory[(stock_node, stock_comp)] = stock
end

# Convert comp_mult_df to Component Multiplier Information
#-----
struct Mults # Mult struct
    loc::String
    component::String
    repair_cost_mult::Float64
    repair_time_mult::Float64
end
mult_dict = Dict() # dictionary of each location and component -> Comp_Mult struct
for df_mults in eachrow(mult_df) # build dictionary
    mults_node = df_mults.node_id
    mults_comp = df_mults.component_id
    repair_cost_mult = df_mults.repair_cost_multiplier # change column name to repair_cost_mult
    repair_time_mult = df_mults.repair_time_multiplier # change column name to repair_time_mult
    mults = Mults(mults_node, mults_comp, repair_cost_mult, repair_time_mult)

```



```

    mult_dict[(mults_node, mults_comp)] = mults
end

```

We then directly implement the Level 3 model in Julia.

```

In [19]: # Parameters
lambda_time = 1 # tradeoff parameters
B = 500         # backorder penalty

# Model
m = Model(Gurobi.Optimizer)

# Variables
@variable(m, x[arc in arcs, comp in components, timestep in 1:timesteps]
           >= 0, Int) # flow x of component through arc
@variable(m, r[node in nodes, comp in components, timestep in 1:timesteps]
           >= 0, Int) # repair r of component at node
@variable(m, s[node in nodes, comp in components, timestep in 0:timesteps]
           >= 0, Int) # store s of component at node
@variable(m, b[node in nodes, comp in components, timestep in 0:timesteps]
           >= 0, Int) # backorder b of component at node

# Constraints
## Flow Conservation
for comp in components, node in nodes, timestep in 1:timesteps
    inflow = sum(x[(node_from, node), comp, timestep]
                 for node_from in incoming[node]) # flow of component into node
    outflow = sum(x[(node, node_to), comp, timestep]
                 for node_to in outgoing[node]) # flow of component out of node
    demand = get(demands, (node, comp), Demand("", "", "", 0, # demand of component present at node
                                                  zeros{Int, timesteps}).time_schedule[timestep] +
                 b[node, comp, timestep - 1] # previos backorders
    backorders = b[node, comp, timestep] # backorders of component at node
    stock_initial = s[node, comp, timestep - 1] # initial stock of component at node
    stock_final = s[node, comp, timestep] # final stock of component at node
    repairs = r[node, comp, timestep] # repairs of component at node
    consumed = sum(qty * r[node, comp_repaired, timestep]
                  for (comp_repaired, qty) in subcomp_dict[comp]; init=0)

```

```

        @constraint(m, stock_initial + inflow + repairs + backorders ==
                    stock_final + outflow + consumed + demand)
    end

    ## Cannot Repair into Part Components
    for comp in components
        if BOM[comp].comp_type == "part"
            for node in nodes, timestep in 1:timesteps
                @constraint(m, r[node, comp, timestep] == 0)
            end
        end
    end

    ## Limit Node Repairs to Repair Capacity Hours
    for node in nodes, timestep in 1:timesteps
        @constraint(m, sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
                        * BOM[comp].repair_time * r[node, comp, timestep] for comp in components)
                    <= node_dict[node].repair_capacity)
    end

    ## Cannot Complete Repairs of Difficulty at Nodes with Insufficient Repair Capability
    for node in nodes, comp in components
        if BOM[comp].repair_difficulty > node_dict[node].repair_capability
            for timestep in 1:timesteps
                @constraint(m, r[node, comp, timestep] == 0)
            end
        end
    end

    ## Initial Storage is Inventory
    for node in nodes, comp in components
        @constraint(m, s[node, comp, 0] == get(inventory, (node, comp), Stock("", "", 0)).qty)
    end

```

```

## Backorders Must be Fulfilled
for node in nodes, comp in components, timestep in [0, timesteps]
    @constraint(m, b[node, comp, timestep] == 0)
end

# Objective
@objective(m, Min,
    sum(arc_dict[arc].trans_cost * x[arc, comp, timestep]
    for comp in components, arc in arcs, timestep in 1:timesteps) + # transport cost
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_cost_mult
        * BOM[comp].repair_cost * r[node, comp, timestep]
        for comp in components, node in nodes, timestep in 1:timesteps) + # repair cost
    sum(B * b[node, comp, timestep]
    for comp in components, node in nodes, timestep in 1:timesteps) + # backorder cost
    lambda_time * (sum(arc_dict[arc].trans_time * x[arc, comp, timestep]
    for comp in components, arc in arcs, timestep in 1:timesteps) + # transport time
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
        * BOM[comp].repair_time * r[node, comp, timestep]
        for comp in components, node in nodes, timestep in 1:timesteps)) # repair time
    )

# Solve
set_silent(m)
optimize!(m)

```

Set parameter Username

Set parameter LicenseID to value 2807856

Academic license - for non-commercial use only - expires 2027-04-15

Results: Level 3

```

In [20]: # Output
println("Optimal Objective Value: ", round(objective_value(m), digits=2))
println()

```

```

## Flow Output
flow_output_df = DataFrame(timestep = Int[],
                           comp_id = String[],
                           node_from = String[],
                           node_to = String[],
                           qty = Int[]
                           )

for comp in components, (node_from, node_to) in arcs, timestep in 1:timesteps # store optimal flow in dataframe
    val = value(x[(node_from, node_to), comp, timestep])
    if val > 1e-3
        push!(flow_output_df, (timestep = timestep,
                                comp_id = comp,
                                node_from = node_from,
                                node_to = node_to,
                                qty = val))
    end
end

sort!(flow_output_df, [:timestep, :comp_id, :node_from, :node_to])
println("Optimal Flow Schedule:") # output optimal flow dataframe
println(flow_output_df)
println()

## Repair Output
repair_output_df = DataFrame(timestep = Int[],
                             comp_id = String[],
                             node_id = String[],
                             qty = Int[]
                             )

for comp in components, node in nodes, timestep in 1:timesteps # store optimal repair in dataframe
    val = value(r[node, comp, timestep])
    if val > 1e-3
        push!(repair_output_df, (timestep = timestep,
                                   comp_id = comp,
                                   node_id = node,
                                   qty = val)
        )
    end
end
end

```

```
sort!(repair_output_df, [:timestep, :comp_id, :node_id,])
println("Optimal Repair Schedule:") # output optimal repairs
println(repair_output_df)
println()

## Backorder Output
backorder_output_df = DataFrame(timestep = Int[],
                                comp_id = String[],
                                node_id = String[],
                                qty = Int[]
                                )
for comp in components, node in nodes, timestep in 1:timesteps # store optimal backorders in dataframe
    val = value(b[node, comp, timestep])
    if val > 1e-3
        push!(backorder_output_df, (timestep = timestep,
                                     comp_id = comp,
                                     node_id = node,
                                     qty = val)
        )
    end
end
sort!(backorder_output_df, [:timestep, :comp_id, :node_id,])
println("Optimal Backorder Schedule:") # output optimal backorders
println(backorder_output_df)

## Convert Output to CSV
mkpath("level3output")
CSV.write("level3output/flow_output.csv", flow_output_df)
CSV.write("level3output/repair_output.csv", repair_output_df)
CSV.write("level3output/backorder_output.csv", backorder_output_df);
```

Optimal Objective Value: 3901.0

Optimal Flow Schedule:

33x5 DataFrame

Row	timestep	comp_id	node_from	node_to	qty
	Int64	String	String	String	Int64
1	1	LRU_2	DEPOT_1	REGWH_1	1
2	1	LRU_2	DEPOT_1	REGWH_2	2
3	1	LRU_2	LOCAL_1	CUST_2	1
4	1	LRU_2	LOCAL_3	CUST_3	2
5	1	LRU_2	LOCAL_4	CUST_4	1
6	1	LRU_2	REGWH_1	LOCAL_1	1
7	1	LRU_2	REGWH_2	LOCAL_3	2
8	1	LRU_2	REGWH_2	LOCAL_4	1
9	1	LRU_2	SPEC_2	DEPOT_1	2
10	1	LRU_2	SPEC_4	REGWH_2	1
11	1	LRU_2_2	DEPOT_1	SPEC_2	2
12	1	LRU_2_2	REGWH_2	SPEC_4	1
13	1	LRU_5	DEPOT_1	REGWH_1	1
14	1	LRU_5	LOCAL_1	CUST_1	1
15	1	LRU_5	REGWH_1	LOCAL_1	1
16	2	LRU_2	DEPOT_1	REGWH_1	1
17	2	LRU_3	DEPOT_1	REGWH_2	1
18	2	LRU_3	LOCAL_3	CUST_3	1
19	2	LRU_3	REGWH_2	LOCAL_3	1
20	2	LRU_5	DEPOT_1	REGWH_1	1
21	2	LRU_5	LOCAL_1	CUST_1	1
22	2	LRU_5	REGWH_1	LOCAL_1	1
23	3	LRU_2	DEPOT_1	REGWH_2	1
24	3	LRU_2	LOCAL_1	CUST_2	1
25	3	LRU_2	LOCAL_3	CUST_3	1
26	3	LRU_2	REGWH_1	LOCAL_1	1
27	3	LRU_2	REGWH_2	LOCAL_3	1
28	3	LRU_5	DEPOT_1	REGWH_1	1
29	3	LRU_5	LOCAL_1	CUST_1	1
30	3	LRU_5	REGWH_1	LOCAL_1	1
31	4	LRU_3	DEPOT_1	REGWH_2	1
32	4	LRU_3	LOCAL_3	CUST_3	1
33	4	LRU_3	REGWH_2	LOCAL_3	1

Optimal Repair Schedule:

9x4 DataFrame

Row	timestep	comp_id	node_id	qty
	Int64	String	String	Int64
1	1	LRU_2	DEPOT_1	1
2	1	LRU_2	SPEC_2	2
3	1	LRU_2	SPEC_4	1
4	1	LRU_5	DEPOT_1	1
5	2	LRU_2	DEPOT_1	1
6	2	LRU_5	DEPOT_1	1
7	3	LRU_2	DEPOT_1	1
8	3	LRU_5	DEPOT_1	1
9	4	LRU_3	DEPOT_1	1

Optimal Backorder Schedule:

0x4 DataFrame

Row	timestep	comp_id	node_id	qty
	Int64	String	String	Int64

Discussion: Level 3

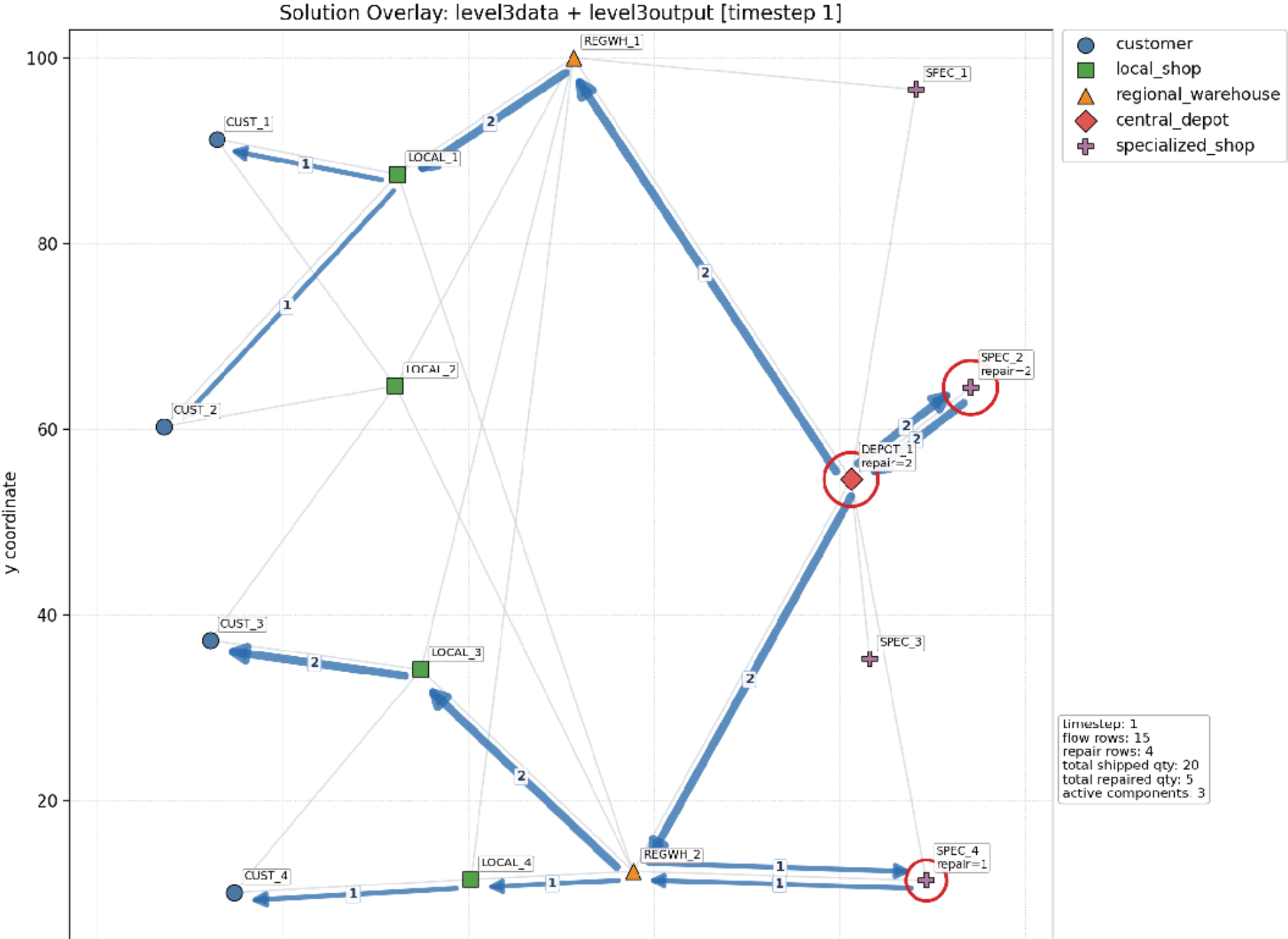
In this level, we introduced separate time slices, which effectively represent distinct actions and demand for each day. A figure for each time slice can be found in `level3output/solution_by_timestep/` . As shown below for time steps 1 and 2, each time step results in a different set of actions to satisfy its corresponding demand.

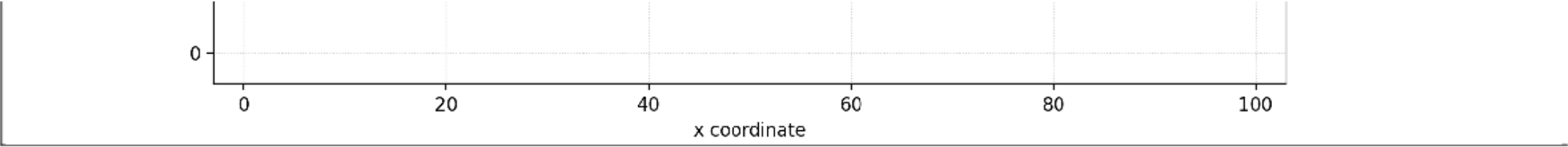
Due to the high cost of backordering, the model chooses to satisfy all demand on its original due date as desired. This behavior aligns with our target application, where delaying aircraft repairs can significantly disrupt flight schedules. However, because the backorder cost is modeled as a tunable parameter, this assumption can be relaxed if desired.

Backordering can also be used in scenarios where it is not possible to meet all demand in a single day due to node-level repair time limits. However, our testing showed that this requires a delicate balance. If demand is increased too much in an attempt to force the model to use backordering, the problem may become infeasible because there are insufficient parts or insufficient repair capacity across the four tested time slices. Future work should extend the model to better handle these cases, perhaps by allowing it to create additional time slices beyond the days for which demand values are explicitly provided or to create new basic parts from scratch at an extreme cost.

```
In [21]: plot(load("./level3output/solution_by_timestep/level3Summary_t1.png"), axis=nothing, size=(1200, 900))
```

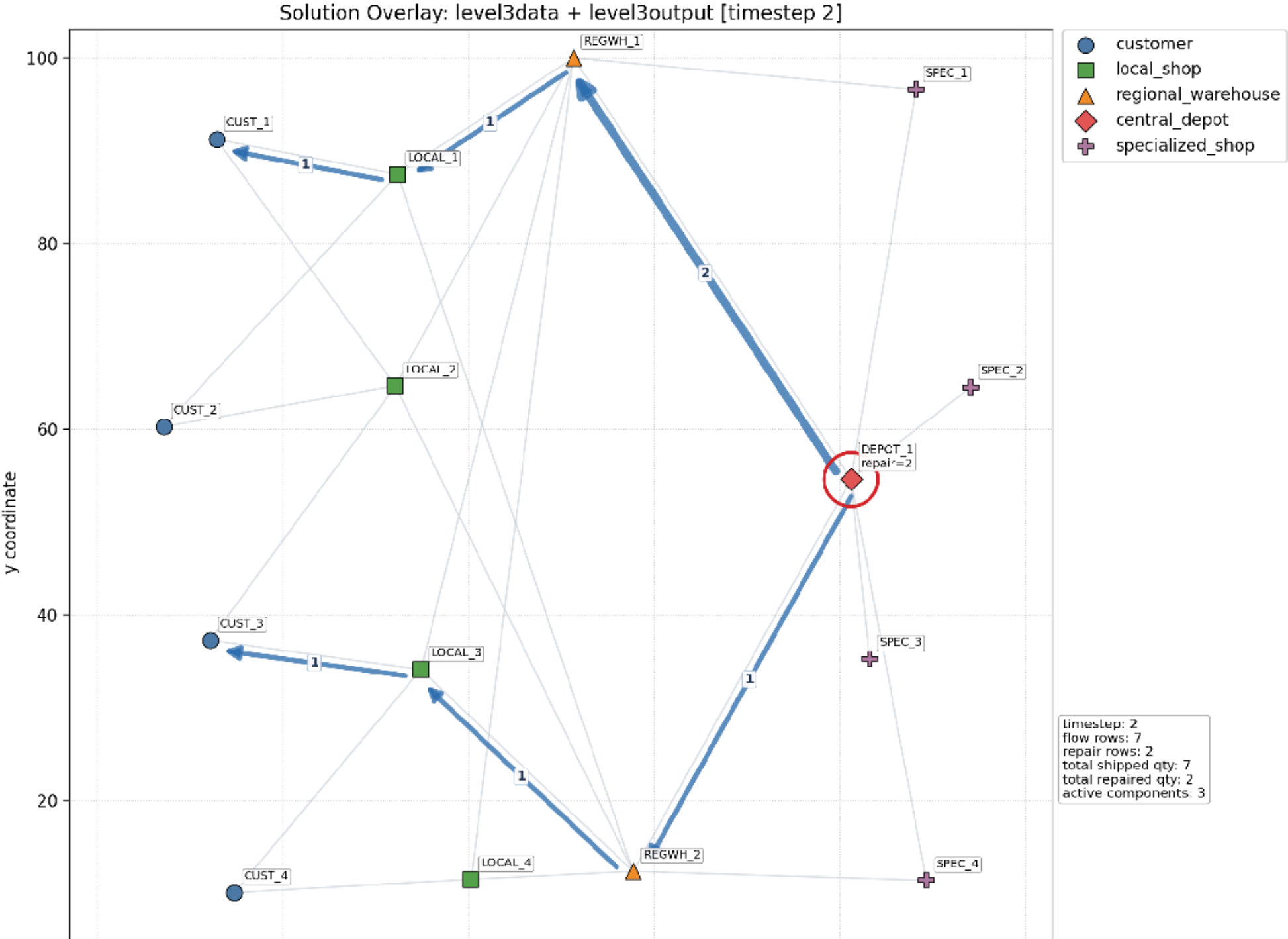
Out[21]:

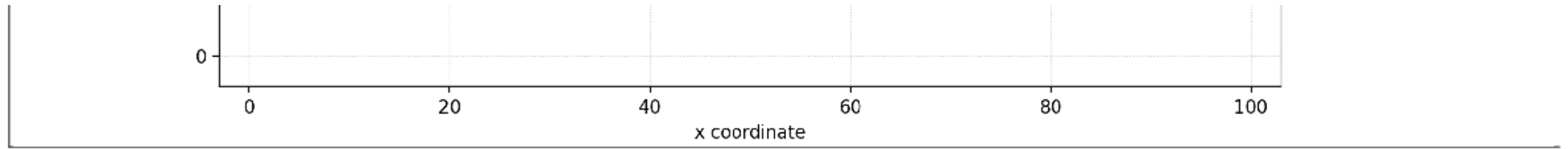




```
In [22]: plot(load("./level3output/solution_by_timestep/level3Summary_t2.png"), axis=nothing, size=(1200, 900))
```

Out[22]:





Below we include another Pareto curve tradeoff analysis. Our results show that in a multi-time period setting, the tradeoff between pure cost and time persists.

```
In [23]: lambdas = sort([0, 0.5, 1., 1.5, 2., 2.5, 3.,
                        3.5, 4., 4.5, 5., 10., 25., 50.]) # tradeoff parameter
opt_costs = []
opt_times = []

# Parameters
B = 500 # backorder penalty

for lambda_time in lambdas
    # Model
    m = Model(Gurobi.Optimizer)

    # Variables
    @variable(m, x[arc in arcs, comp in components, timestep in 1:timesteps]
              >= 0, Int) # flow x of component through arc
    @variable(m, r[node in nodes, comp in components, timestep in 1:timesteps]
              >= 0, Int) # repair r of component at node
    @variable(m, s[node in nodes, comp in components, timestep in 0:timesteps]
              >= 0, Int) # store s of component at node
    @variable(m, b[node in nodes, comp in components, timestep in 0:timesteps]
              >= 0, Int) # backorder b of component at node

    # Constraints
    ## Flow Conservation
```

```

for comp in components, node in nodes, timestep in 1:timesteps
    inflow = sum(x[(node_from, node), comp, timestep]           # flow of component into node
                for node_from in incoming[node])
    outflow = sum(x[(node, node_to), comp, timestep]           # flow of component out of node
                for node_to in outgoing[node])
    demand = get(demands, (node, comp), Demand("", "", "", 0, # demand of component present at node
                zeros(Int, timesteps))).time_schedule[timestep] +
                b[node, comp, timestep - 1]                   # previos backorders
    backorders = b[node, comp, timestep]                       # backorders of component at node
    stock_initial = s[node, comp, timestep - 1]               # initial stock of component at node
    stock_final = s[node, comp, timestep]                     # final stock of component at node
    repairs = r[node, comp, timestep]                         # repairs of component at node
    consumed = sum(qty * r[node, comp_repaired, timestep]     # component consumed for repairs at node
                for (comp_repaired, qty) in subcomp_dict[comp]; init=0)
    @constraint(m, stock_initial + inflow + repairs + backorders ==
                stock_final + outflow + consumed + demand)
end

## Cannot Repair into Part Components
for comp in components
    if BOM[comp].comp_type == "part"
        for node in nodes, timestep in 1:timesteps
            @constraint(m, r[node, comp, timestep] == 0)
        end
    end
end

## Limit Node Repairs to Repair Capacity Hours
for node in nodes, timestep in 1:timesteps
    @constraint(m, sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
                * BOM[comp].repair_time * r[node, comp, timestep] for comp in components)
                <= node_dict[node].repair_capacity)
end

## Cannot Complete Repairs of Difficulty at Nodes with Insufficient Repair Capability
for node in nodes, comp in components

```

```

    if BOM[comp].repair_difficulty > node_dict[node].repair_capability
        for timestep in 1:timesteps
            @constraint(m, r[node, comp, timestep] == 0)
        end
    end
end

end

## Initial Storage is Inventory
for node in nodes, comp in components
    @constraint(m, s[node, comp, 0] == get(inventory, (node, comp), Stock("", "", 0)).qty)
end

## Backorders Must be Fulfilled
for node in nodes, comp in components, timestep in [0, timesteps]
    @constraint(m, b[node, comp, timestep] == 0)
end

# Objective
@objective(m, Min,
    sum(arc_dict[arc].trans_cost * x[arc, comp, timestep]
    for comp in components, arc in arcs, timestep in 1:timesteps) + # transport cost
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_cost_mult
        * BOM[comp].repair_cost * r[node, comp, timestep]
    for comp in components, node in nodes, timestep in 1:timesteps) + # repair cost
    sum(B * b[node, comp, timestep]
    for comp in components, node in nodes, timestep in 1:timesteps) + # backorder cost
    lambda_time * (sum(arc_dict[arc].trans_time * x[arc, comp, timestep]
    for comp in components, arc in arcs, timestep in 1:timesteps) + # transport time
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
        * BOM[comp].repair_time * r[node, comp, timestep]
    for comp in components, node in nodes, timestep in 1:timesteps)) # repair time
    )

# Solve

```

```

set_silent(m)
optimize!(m)

# Extract Values
opt_cost = sum(arc_dict[arc].trans_cost * value(x[arc, comp, timestep])
              for comp in components, arc in arcs, timestep in 1:timesteps) + # transport cost
              sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_cost_mult
                  * BOM[comp].repair_cost * value(r[node, comp, timestep])
                  for comp in components, node in nodes, timestep in 1:timesteps) + # repair cost
              sum(500 * value(b[node, comp, timestep])
                  for comp in components, node in nodes, timestep in 1:timesteps) # backorder cost
opt_time = sum(arc_dict[arc].trans_time * value(x[arc, comp, timestep])
              for comp in components, arc in arcs, timestep in 1:timesteps) + # transport time
              sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
                  * BOM[comp].repair_time * value(r[node, comp, timestep])
                  for comp in components, node in nodes, timestep in 1:timesteps) # repair time
push!(opt_costs, opt_cost)
push!(opt_times, opt_time)
end

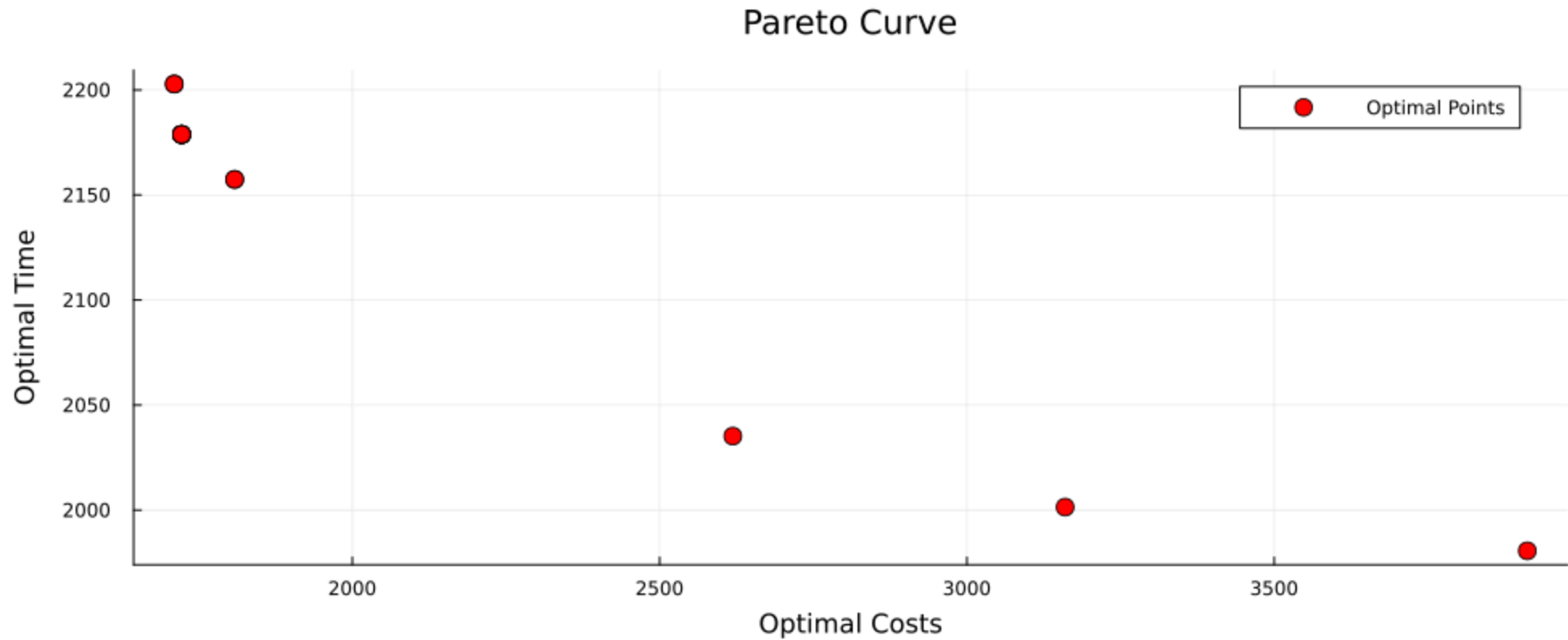
# Plot Pareto Curve
p = scatter(opt_costs, opt_times, marker=:circle, markersize=6,
            label="Optimal Points", color=:red)
title!(p, "Pareto Curve")
xlabel!(p, "Optimal Costs")
ylabel!(p, "Optimal Time")

# Display
plot(p, size=(1000, 400), margin=5Plots.mm)

```

[illegible]

Out[23]:



Level 3.5: Math Model, Implementation, Results, and Discussion

Level 3.5 modifies Level 3 by rebuilding the transportation cost and time formulation. In the previous levels, each component transported across an arc incurred its own separate transportation cost and time penalty. For example, if two units of the same component were shipped across the same arc, the model added the transportation cost and time penalty twice. As we previously discussed, this assumption is unrealistic because most transportation modes, such as trucks, trains, and planes, can carry multiple components at the same time.

To account for this, we model transportation using both a fixed cost and a variable cost across both price and time. The fixed cost represents the upfront cost of using a transportation mode, such as leasing a vehicle, paying an operator, or purchasing a transportation service, while the fixed time represents the baseline travel time required to move across the arc. We then add a linear variable cost based on the number of components transported. This captures the additional marginal cost of moving more components, such as increased fuel usage, loading and unloading labor, or handling costs.

To capture this bulk shipping policy, we introduce an indicator variable for each arc and time step. This variable is set to 1 if any component is transported across the arc during that time slice, and 0 otherwise. If so, then we add the arc's transportation cost and time to the objective function. The additional linear cost is implemented using a simplified version of the previous transportation cost formulation. Specifically, we apply the original per-component transportation cost with a scaling factor, so that the marginal cost of adding components remains small relative to the fixed bulk-shipping cost. Notably, we do not add any additional transportation time for moving more components. Since the model allows at most one transportation event per arc in each time slice, the travel time is incurred only once. As a result, this formulation makes bulk transportation more expensive in terms of cost than time.

To implement this logic, we use a standard Big- M formulation. If the indicator variable is 0, no transportation is allowed across the associated arc. If the indicator variable is 1, transportation across that arc is permitted. We choose M to be the total initial inventory of all components across the entire network, including LRUs, SRUs, and parts. While this choice is intuitive because it provides an upper bound on the amount that could be transported, it is likely overly conservative. In practice, transportation across a single arc is much smaller than the total number of components in the network, so this value of M is unnecessarily large and computationally inefficient.

Math Model: Level 3.5

We want to modify our Level 3 math model to include the alternate logic of Level 3.5. Specifically, want to convert the transportation cost and time logic so that, for each timestep, we bulk transport components along nodes with a large initial cost/time penalty and then marginal cost increases with components transported.

Additional Problem Data

Let $\mathcal{T}$ denote the set of timesteps (e.g. days).

Let $R \geq 0$ denote the marginal transport cost of transporting components over the arc relative to the arcs cost; e.g. $R = \frac{1}{10}$ means moving one additional component along the arc has a cost of $\frac{1}{10}$ of the standard transport cost; let M be solve large parameter such that the following $y_{a,t}$ binary decision variables enact the desired fixed cost.

Additional Decision Variables and Variable Modifications

Let $y_{a,t} \in \{0, 1\}$ denote if any $x_{a,k,t} > 0$ over $k \in K$.

Additional Constraints

We want $x_{a,k,t} > 0$ only if $y_{a,t} = 1$ for any $k \in K$,

$$x_{a,k,t} \leq M y_{a,t}. \qquad a \in A, k \in K, t \in \mathcal{T}$$

Objective Function Modifications

We want to make transportation cost be a fixed cost if any components are transported across a given arc as adding the sum, over all arcs and each time-slice, of the cost of transportation across the arc only if the indicator variable is 1 for the arc in the time-slice,

$$\sum_{a \in A, t \in \mathcal{T}} C_a^{flow} y_{a,t},$$

and also a marginal cost for each additional component transported across the arc as the sum, over each arc and each time-slice, of components transported over an arc multiplied by the transportation cost of transportation over that arc scaled by the marginal constant C ,

$$\sum_{a \in A, k \in K, t \in \mathcal{T}} RC_a^{flow} x_{a,k,t};$$

thus, we have our full cost objective,

$$\left[\sum_{a \in A, t \in \mathcal{T}} C_a^{flow} y_{a,t} + \sum_{a \in A, k \in K, t \in \mathcal{T}} RC_a^{flow} x_{a,k,t} + \sum_{n \in N, k \in K, t \in \mathcal{T}} m_n^{\text{cost}} C_k^{\text{repair}} r_{n,k,t} + \sum_{n \in N, k \in K, t \in \mathcal{T}} Bb_{n,k,t} \right].$$

We want to make transportation time be a fixed time if any components are transported across a given arc. We sum the transportation time over each arc and time-slice only if the indicator variable is 1.

$$\sum_{a \in A, t \in \mathcal{T}} T_a^{flow} y_{a,t};$$

thus, we have our full time objective,

$$\sum_{a \in A, t \in \mathcal{T}} T_a^{flow} y_{a,t} + \sum_{n \in N, k \in K, t \in \mathcal{T}} m_n^{\text{time}} T_k^{\text{repair}} r_{n,k,t}$$

Putting this together, we have the integer linear program,

$$\begin{aligned} \min \quad & \left[\left(\sum_{a \in A, t \in \mathcal{T}} C_a^{flow} y_{a,t} + \sum_{a \in A, k \in K, t \in \mathcal{T}} RC_a^{flow} x_{a,k,t} + \right. \right. \\ & \left. \sum_{n \in N, k \in K, t \in \mathcal{T}} m_n^{cost} C_k^{repair} r_{n,k,t} + \sum_{n \in N, k \in K, t \in \mathcal{T}} B b_{n,k,t} \right) + \\ & \left. \lambda^{time} \left(\sum_{a \in A, t \in \mathcal{T}} T_a^{flow} y_{a,t} + \sum_{n \in N, k \in K, t \in \mathcal{T}} m_n^{time} T_k^{repair} r_{n,k,t} \right) \right] \\ \text{s.t.} \quad & s_{n,k,t-1} + \sum_{(i,n) \in A} x_{(i,n),k,t} + r_{n,k,t} + b_{n,k,t} = \\ & s_{n,k,t} + \sum_{(n,i) \in A} x_{(n,i),k,t} + \sum_{k' \in K_k} r_{n,k',t} q_{k',k} + d_{n,k}, & n \in N, k \in K, t \in \mathcal{T} \\ & r_{n,k,t} = 0, & n \in N \setminus N_k, k \in K, t \in \mathcal{T} \\ & \sum_{k \in K} m_n^{time} T_k^{repair} r_{n,k,t} \leq T_n^{cap}, & n \in N, t \in \mathcal{T} \\ & x_{a,k,t} \leq M y_{a,t}, & a \in A, k \in K, t \in \mathcal{T} \\ & x_{a,k,t} \geq 0, & a \in A, k \in K, t \in \mathcal{T} \\ & r_{n,k,t} \geq 0, & n \in N, k \in K, t \in \mathcal{T} \\ & s_{n,k,t} \geq 0, & n \in N, k \in K, t \in \mathcal{T} \\ & b_{n,k,t} \geq 0, & n \in N, k \in K, t \in \mathcal{T} \\ & y_{a,t} \in \{0, 1\}. & a \in A, t \in \mathcal{T} \end{aligned}$$

Implementation: Level 3.5

We use the same data as in the Level 3 model.

```
In [24]: M = sum(node_inv_df.quantity); # generate Big-M for Flow
```

```
In [25]: # Parameters
lambda_time = 1 # tradeoff parameters
B = 500         # backorder penalty
R = 1 / 10      # marginal transport price ratio
```

```

# Model
m = Model(Gurobi.Optimizer)

# Variables
@variable(m, x[arc in arcs, comp in components, timestep in 1:timesteps]
           >= 0, Int)           # flow x of component through arc
@variable(m, y[arc in arcs, timestep in 1:timesteps], Bin) # y gives if flow through arc
@variable(m, r[node in nodes, comp in components, timestep in 1:timesteps]
           >= 0, Int)           # repair r of component at node
@variable(m, s[node in nodes, comp in components, timestep in 0:timesteps]
           >= 0, Int)           # store s of component at node
@variable(m, b[node in nodes, comp in components, timestep in 0:timesteps]
           >= 0, Int)           # backorder b of component at node

# Constraints
## Flow Conservation
for comp in components, node in nodes, timestep in 1:timesteps
    inflow = sum(x[(node_from, node), comp, timestep]
                 for node_from in incoming[node]) # flow of component into node
    outflow = sum(x[(node, node_to), comp, timestep]
                  for node_to in outgoing[node]) # flow of component out of node
    demand = get(demands, (node, comp), Demand("", "", "", 0, # demand of component present at node
                                                    zeros(Int, timesteps))).time_schedule[timestep] +
              b[node, comp, timestep - 1] # previos backorders
    backorders = b[node, comp, timestep] # backorders of component at node
    stock_initial = s[node, comp, timestep - 1] # initial stock of component at node
    stock_final = s[node, comp, timestep] # final stock of component at node
    repairs = r[node, comp, timestep] # repairs of component at node
    consumed = sum(qty * r[node, comp_repaired, timestep]
                   for (comp_repaired, qty) in subcomp_dict[comp]; init=0) # component consumed for repairs at node
    @constraint(m, stock_initial + inflow + repairs + backorders ==
                stock_final + outflow + consumed + demand)
end

## Cannot Repair into Part Components

```

```

for comp in components
  if BOM[comp].comp_type == "part"
    for node in nodes, timestep in 1:timesteps
      @constraint(m, r[node, comp, timestep] == 0)
    end
  end
end

## Cannot Complete Repairs of Difficulty at Nodes with Insufficient Repair Capability
for node in nodes, comp in components
  if BOM[comp].repair_difficulty > node_dict[node].repair_capability
    for timestep in 1:timesteps
      @constraint(m, r[node, comp, timestep] == 0)
    end
  end
end

## Limit Node Repairs to Repair Capacity Hours
for node in nodes, timestep in 1:timesteps
  @constraint(m, sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
    * BOM[comp].repair_time * r[node, comp, timestep] for comp in components)
    <= node_dict[node].repair_capacity)
end

## Initial Storage is Inventory
for node in nodes, comp in components
  @constraint(m, s[node, comp, 0] == get(inventory, (node, comp), Stock("", "", 0)).qty)
end

## Backorders Must be Fulfilled
for node in nodes, comp in components, timestep in [0, timesteps]
  @constraint(m, b[node, comp, timestep] == 0)
end

```

```

## Transport Decision y Constraint
for arc in arcs, comp in components, timestep in 1:timesteps
    @constraint(m, x[arc, comp, timestep] <= M * y[arc, timestep])
end

# Objective
@objective(m, Min,
    sum(arc_dict[arc].trans_cost * y[arc, timestep]
        for arc in arcs, timestep in 1:timesteps) + # transport cost base
    sum(R * arc_dict[arc].trans_cost * x[arc, comp, timestep]
        for arc in arcs, comp in components, timestep in 1:timesteps) + # transport cost marginal
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_cost_mult
        * BOM[comp].repair_cost * r[node, comp, timestep]
        for comp in components, node in nodes, timestep in 1:timesteps) + # repair cost
    sum(B * b[node, comp, timestep]
        for comp in components, node in nodes, timestep in 1:timesteps) + # backorder cost
    lambda_time * (sum(arc_dict[arc].trans_time * y[arc, timestep]
        for arc in arcs, timestep in 1:timesteps) + # transport time
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
        * BOM[comp].repair_time * r[node, comp, timestep]
        for comp in components, node in nodes, timestep in 1:timesteps)) # repair time
    )

# Solve
set_silent(m)
optimize!(m)

```

Set parameter Username

Set parameter LicenseID to value 2807856

Academic license - for non-commercial use only - expires 2027-04-15

```

In [26]: # Parameters
lambda_time = 1 # tradeoff parameters
B = 500         # backorder penalty
C = 1 / 10      # marginal transport price ratio
timesteps=4

```

```

# Model
m = Model(Gurobi.Optimizer)

# Variables
@variable(m, x[arc in arcs, comp in components, timestep in 1:timesteps]
           >= 0, Int) # flow x of component through arc
@variable(m, y[arc in arcs, timestep in 1:timesteps], Bin) # y gives if flow through arc
@variable(m, r[node in nodes, comp in components, timestep in 1:timesteps]
           >= 0, Int) # repair r of component at node
@variable(m, s[node in nodes, comp in components, timestep in 0:timesteps]
           >= 0, Int) # store s of component at node
@variable(m, b[node in nodes, comp in components, timestep in 0:timesteps]
           >= 0, Int) # backorder b of component at node

# Constraints
## Flow Conservation
for comp in components, node in nodes, timestep in 1:timesteps
    inflow = sum(x[(node_from, node), comp, timestep] # flow of component into node
                 for node_from in incoming[node])
    outflow = sum(x[(node, node_to), comp, timestep] # flow of component out of node
                  for node_to in outgoing[node])
    demand = get(demands, (node, comp), Demand("", "", "", 0, # demand of component present at node
                                                  zeros(Int, timesteps))).time_schedule[timestep] +
              b[node, comp, timestep - 1] # previos backorders
    backorders = b[node, comp, timestep] # backorders of component at node
    stock_initial = s[node, comp, timestep - 1] # initial stock of component at node
    stock_final = s[node, comp, timestep] # final stock of component at node
    repairs = r[node, comp, timestep] # repairs of component at node
    consumed = sum(qty * r[node, comp_repaired, timestep] # component consumed for repairs at node
                   for (comp_repaired, qty) in subcomp_dict[comp]; init=0)
    @constraint(m, stock_initial + inflow + repairs + backorders ==
                stock_final + outflow + consumed + demand)
end

```

```

## Cannot Repair into Part Components
for comp in components
    if BOM[comp].comp_type == "part"
        for node in nodes, timestep in 1:timesteps
            @constraint(m, r[node, comp, timestep] == 0)
        end
    end
end

## Limit Node Repairs to Repair Capacity Hours
for node in nodes, timestep in 1:timesteps
    @constraint(m, sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
        * BOM[comp].repair_time * r[node, comp, timestep] for comp in components)
        <= node_dict[node].repair_capacity)
end

## Cannot Complete Repairs of Difficulty at Nodes with Insufficient Repair Capability
for node in nodes, comp in components
    if BOM[comp].repair_difficulty > node_dict[node].repair_capability
        for timestep in 1:timesteps
            @constraint(m, r[node, comp, timestep] == 0)
        end
    end
end

## Initial Storage is Inventory
for node in nodes, comp in components
    @constraint(m, s[node, comp, 0] == get(inventory, (node, comp), Stock("", "", 0)).qty)
end

## Backorders Must be Fulfilled
for node in nodes, comp in components, timestep in [0, timesteps]
    @constraint(m, b[node, comp, timestep] == 0)
end

```


end

Transport Decision y Constraint

```
for arc in arcs, comp in components, timestep in 1:timesteps
    @constraint(m, x[arc, comp, timestep] <= M * y[arc, timestep])
end
```

No Early Fulfillment at Customer Nodes

```
# for node in nodes, comp in components, timestep in 1:timesteps
#     if node_dict[node].node_type == "customer"
#         @constraint(m, s[node, comp, timestep] == 0)
#     end
# end
```

Objective

```
@objective(m, Min,
    sum(arc_dict[arc].trans_cost * y[arc, timestep]
        for arc in arcs, timestep in 1:timesteps) + # transport cost base
    sum(C * arc_dict[arc].trans_cost * x[arc, comp, timestep]
        for arc in arcs, comp in components, timestep in 1:timesteps) + # transport cost marginal
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_cost_mult
        * BOM[comp].repair_cost * r[node, comp, timestep]
        for comp in components, node in nodes, timestep in 1:timesteps) + # repair cost
    sum(B * b[node, comp, timestep]
        for comp in components, node in nodes, timestep in 1:timesteps) + # backorder cost
    lambda_time * (sum(arc_dict[arc].trans_time * y[arc, timestep]
        for arc in arcs, timestep in 1:timesteps) + # transport time
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
        * BOM[comp].repair_time * r[node, comp, timestep]
        for comp in components, node in nodes, timestep in 1:timesteps)) # repair time
)
```

Solve

```
set_silent(m)
optimize!(m)
```

Set parameter Username

Set parameter LicenseID to value 2807856

Academic license - for non-commercial use only - expires 2027-04-15

Solution: Level 3.5

```
In [27]: # Output
println("Optimal Objective Value: ", round(objective_value(m), digits=2))
println()

## Flow Output
flow_output_df = DataFrame(timestep = Int[],
                           comp_id = String[],
                           node_from = String[],
                           node_to = String[],
                           qty = Int[]
                           )
for comp in components, (node_from, node_to) in arcs, timestep in 1:timesteps # store optimal flow in dataframe
    val = value(x[(node_from, node_to), comp, timestep])
    if val > 1e-3
        push!(flow_output_df, (timestep = timestep,
                                comp_id = comp,
                                node_from = node_from,
                                node_to = node_to,
                                qty = val))
    end
end
sort!(flow_output_df, [:timestep, :comp_id, :node_from, :node_to])
println("Optimal Flow Schedule:") # output optimal flow dataframe
println(flow_output_df)
println()

## Repair Output
repair_output_df = DataFrame(timestep = Int[],
```

```

        comp_id = String[],
        node_id = String[],
        qty = Int[]
    )
for comp in components, node in nodes, timestep in 1:timesteps # store optimal repair in dataframe
    val = value(r[node, comp, timestep])
    if val > 1e-3
        push!(repair_output_df, (timestep = timestep,
                                comp_id = comp,
                                node_id = node,
                                qty = val)
        )
    end
end
sort!(repair_output_df, [:timestep, :comp_id, :node_id,])
println("Optimal Repair Schedule:") # output optimal repairs
println(repair_output_df)
println()

## Backorder Output
backorder_output_df = DataFrame(timestep = Int[],
                                comp_id = String[],
                                node_id = String[],
                                qty = Int[]
                                )
for comp in components, node in nodes, timestep in 1:timesteps # store optimal backorders in dataframe
    val = value(b[node, comp, timestep])
    if val > 1e-3
        push!(backorder_output_df, (timestep = timestep,
                                    comp_id = comp,
                                    node_id = node,
                                    qty = val)
        )
    end
end
sort!(backorder_output_df, [:timestep, :comp_id, :node_id,])
println("Optimal Backorder Schedule:") # output optimal backorders
println(backorder_output_df)

```

```
## Convert Output to CSV
mkpath("level3output")
CSV.write("level3_5output/flow_output.csv", flow_output_df)
CSV.write("level3_5output/repair_output.csv", repair_output_df)
CSV.write("level3_5output/backorder_output.csv", backorder_output_df);
```

Optimal Objective Value: 2152.8

Optimal Flow Schedule:

20x5 DataFrame

Row	timestep	comp_id	node_from	node_to	qty
	Int64	String	String	String	Int64
1	1	LRU_2	CUST_3	LOCAL_3	1
2	1	LRU_2	DEPOT_1	SPEC_1	1
3	1	LRU_2	LOCAL_2	CUST_2	2
4	1	LRU_2	LOCAL_2	CUST_3	4
5	1	LRU_2	LOCAL_3	CUST_4	1
6	1	LRU_2	REGWH_1	LOCAL_2	6
7	1	LRU_2	SPEC_1	REGWH_1	6
8	1	LRU_2_1	DEPOT_1	SPEC_1	4
9	1	LRU_2_2	DEPOT_1	SPEC_1	5
10	1	LRU_3	DEPOT_1	SPEC_1	2
11	1	LRU_3	LOCAL_2	CUST_3	2
12	1	LRU_3	REGWH_1	LOCAL_2	2
13	1	LRU_3	SPEC_1	REGWH_1	2
14	1	LRU_5	DEPOT_1	SPEC_1	3
15	1	LRU_5	LOCAL_2	CUST_1	3
16	1	LRU_5	REGWH_1	LOCAL_2	3
17	1	LRU_5	SPEC_1	REGWH_1	3
18	1	LRU_5	SPEC_3	DEPOT_1	3
19	1	LRU_5_1	DEPOT_1	SPEC_3	6
20	1	LRU_5_2	DEPOT_1	SPEC_3	2

Optimal Repair Schedule:

4x4 DataFrame

Row	timestep	comp_id	node_id	qty
	Int64	String	String	Int64
1	1	LRU_2	DEPOT_1	1
2	1	LRU_2	SPEC_1	5
3	1	LRU_3	DEPOT_1	1
4	1	LRU_5	SPEC_3	3

Optimal Backorder Schedule:

0x4 DataFrame

Row	timestep	comp_id	node_id	qty
-----	----------	---------	---------	-----

	Int64	String	String	Int64
--	-------	--------	--------	-------

Discussion: Level 3.5

As expected, batching orders significantly reduces total cost, dropping the objective from 3901 in Level 3 to 2152.8 in Level 3.5. Notably, all demand is fulfilled on the first day. To understand this behavior, we analyzed the model more closely and found that the Level 3.5 formulation creates an incentive to consolidate shipments. Because each route incurs a fixed cost when used, the model prefers to traverse a route as few times as possible. In this small example, the number of components and customer demands is limited enough that the repair nodes can complete all required work on the first day without exceeding their capacity. In principle, this is not necessarily incorrect: it reflects the cost advantage of batching shipments when sufficient repair capacity is available. Therefore, we leave this behavior in the model and note that, in larger-scale examples, shop capacity would likely prevent all repairs from being completed immediately.

If users want to prevent early fulfillment, they can comment the constraint titled "No Early Fulfillment at Customer Nodes", which prevents repaired units from being delivered to customers before their requested day.

The Pareto curve analysis displays behavior similar to the Level 3 model, so we omit further discussion.

```
In [28]: lambdas = sort([0, 0.5, 1., 1.5, 2., 2.5, 3., 3.5, 4., 4.5, 5.]) # tradeoff parameter
opt_costs = []
opt_times = []

In [29]: # Parameters
B = 500 # backorder penalty
C = 1 / 10 # marginal transport price ratio

for lambda_time in lambdas
    # Model
    m = Model(Gurobi.Optimizer)

    # Variables
    @variable(m, x[arc in arcs, comp in components, timestep in 1:timesteps]
              >= 0, Int) # flow x of component through arc
    @variable(m, y[arc in arcs, timestep in 1:timesteps], Bin) # y gives if flow through arc
    @variable(m, r[node in nodes, comp in components, timestep in 1:timesteps]
              >= 0, Int) # repair r of component at node
    @variable(m, s[node in nodes, comp in components, timestep in 0:timesteps]
              >= 0, Int) # store s of component at node
```

```

@variable(m, b[node in nodes, comp in components, timestep in 0:timesteps]
          >= 0, Int)                                     # backorder b of component at node

# Constraints
## Flow Conservation
for comp in components, node in nodes, timestep in 1:timesteps
    inflow = sum(x[(node_from, node), comp, timestep]      # flow of component into node
                 for node_from in incoming[node])
    outflow = sum(x[(node, node_to), comp, timestep]        # flow of component out of node
                 for node_to in outgoing[node])
    demand = get(demands, (node, comp), Demand("", "", "", 0, # demand of component present at node
                                                  zeros(Int, timesteps))).time_schedule[timestep] +
              b[node, comp, timestep - 1]                  # previos backorders
    backorders = b[node, comp, timestep]                    # backorders of component at node
    stock_initial = s[node, comp, timestep - 1]             # initial stock of component at node
    stock_final = s[node, comp, timestep]                   # final stock of component at node
    repairs = r[node, comp, timestep]                       # repairs of component at node
    consumed = sum(qty * r[node, comp_repaired, timestep]   # component consumed for repairs at node
                  for (comp_repaired, qty) in subcomp_dict[comp]; init=0)
    @constraint(m, stock_initial + inflow + repairs + backorders ==
               stock_final + outflow + consumed + demand)
end

## Cannot Repair into Part Components
for comp in components
    if BOM[comp].comp_type == "part"
        for node in nodes, timestep in 1:timesteps
            @constraint(m, r[node, comp, timestep] == 0)
        end
    end
end

## Limit Node Repairs to Repair Capacity Hours
for node in nodes, timestep in 1:timesteps
    @constraint(m, sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
                      * BOM[comp].repair_time * r[node, comp, timestep] for comp in components)
               <= node_dict[node].repair_capacity)
end

```

```

## Cannot Complete Repairs of Difficulty at Nodes with Insufficient Repair Capability
for node in nodes, comp in components
    if BOM[comp].repair_difficulty > node_dict[node].repair_capability
        for timestep in 1:timesteps
            @constraint(m, r[node, comp, timestep] == 0)
        end
    end
end

## Initial Storage is Inventory
for node in nodes, comp in components
    @constraint(m, s[node, comp, 0] == get(inventory, (node, comp), Stock("", "", 0)).qty)
end

## Backorders Must be Fulfilled
for node in nodes, comp in components, timestep in [0, timesteps]
    @constraint(m, b[node, comp, timestep] == 0)
end

## Transport Decision y Constraint
for arc in arcs, comp in components, timestep in 1:timesteps
    @constraint(m, x[arc, comp, timestep] <= M * y[arc, timestep])
end

# Objective
@objective(m, Min,
    sum(arc_dict[arc].trans_cost * y[arc, timestep]
        for arc in arcs, timestep in 1:timesteps) + # transport cost base
    sum(C * arc_dict[arc].trans_cost * x[arc, comp, timestep]
        for arc in arcs, comp in components, timestep in 1:timesteps) + # transport cost marginal
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_cost_mult
        * BOM[comp].repair_cost * r[node, comp, timestep]
        for comp in components, node in nodes, timestep in 1:timesteps) + # repair cost
    sum(B * b[node, comp, timestep]
        for comp in components, node in nodes, timestep in 1:timesteps) + # backorder cost
    lambda_time * (sum(arc_dict[arc].trans_time * y[arc, timestep]
        for arc in arcs, timestep in 1:timesteps) + # transport time
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
        * BOM[comp].repair_time * r[node, comp, timestep]
        for comp in components, node in nodes, timestep in 1:timesteps)) # repair time

```



```

    )

# Solve
set_silent(m)
optimize!(m)

# Extract Values
opt_cost = sum(arc_dict[arc].trans_cost * value(y[arc, timestep])
               for arc in arcs, timestep in 1:timesteps) + # transport cost base
sum(C * arc_dict[arc].trans_cost * value(x[arc, comp, timestep])
    for arc in arcs, comp in components, timestep in 1:timesteps) + # transport cost marginal
sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_cost_mult
    * BOM[comp].repair_cost * value(r[node, comp, timestep])
    for comp in components, node in nodes, timestep in 1:timesteps) + # repair cost
sum(B * value(b[node, comp, timestep])
    for comp in components, node in nodes, timestep in 1:timesteps) # backorder cost
opt_time = sum(arc_dict[arc].trans_time * value(y[arc, timestep])
               for arc in arcs, timestep in 1:timesteps) + # transport time
sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
    * BOM[comp].repair_time * value(r[node, comp, timestep])
    for comp in components, node in nodes, timestep in 1:timesteps) # repair time
push!(opt_costs, opt_cost)
push!(opt_times, opt_time)
end

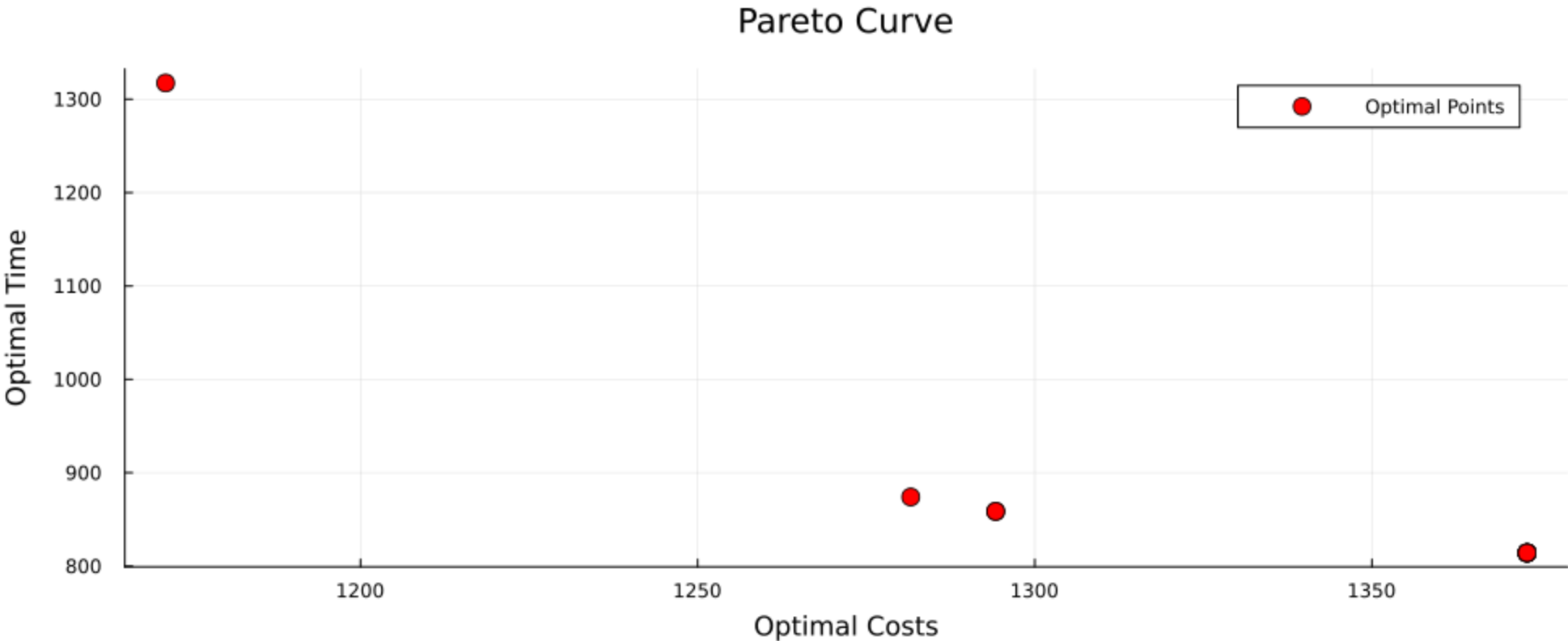
```

[illegible]

```
In [30]: # Plot Pareto Curve
p = scatter(opt_costs, opt_times, marker=:circle, markersize=6,
            label="Optimal Points", color=:red)
title!(p, "Pareto Curve")
xlabel!(p, "Optimal Costs")
ylabel!(p, "Optimal Time")
```

```
# Display
plot(p, size=(1000, 400), margin=5Plots.mm)
```

Out[30]:



Testing a Multi-Day Example

In this section, we confirm our intuition that at a slightly larger scale, the model is forced to use multiple days to fulfill orders. Additionally, we test the model's efficiency on a standard consumer laptop (Dell XPS-13). We increase the number of components to 40, and we found that the model now spreads the repairs across two time periods. However, even though we only increased the number of components from 36 to 40, we noticed a significant jump in the time required to finish optimizing our model. This suggests that our model's computational efficiency is very sensitive to the total number of components. The high computational cost is likely influenced by the Big- M bound being significantly larger than necessary. Future work could explore more fine-grained approaches to select tighter M values.

```
In [31]: # Load Data
node_df = CSV.read("bigger_level3data/nodes.csv", DataFrame,delim=',')
arc_df = CSV.read("bigger_level3data/arcs.csv", DataFrame,delim=',')
```

```
bom_df = CSV.read("bigger_level3data/bom.csv", DataFrame,delim=',')
demand_df = CSV.read("bigger_level3data/demand.csv", DataFrame,delim=',')
node_inv_df = CSV.read("bigger_level3data/node_inventory.csv", DataFrame,delim=',')
mult_df = CSV.read("bigger_level3data/component_multiplier.csv", DataFrame,delim=',')
```

```
# Convert node_df to Node Information
```

```
#-----
```

```
struct Node # Node struct
```

```
  name::String
```

```
  node_type::String
```

```
  loc::Tuple{Float64, Float64}
```

```
  repair_capability::Int
```

```
  repair_capacity::Int
```

```
end
```

```
node_dict = Dict() # dictionary of nodes -> Node struct
```

```
for df_node in eachrow(node_df) # add nodes to node_dict
```

```
  node_id = df_node.node_id
```

```
  node_type = df_node.node_type
```

```
  (x, y) = (df_node.x, df_node.y)
```

```
  repair_capability = df_node.repair_capability
```

```
  repair_capacity = df_node.repair_capacity
```

```
  node = Node(node_id, node_type, (x, y), repair_capability, repair_capacity)
```

```
  node_dict[node_id] = node
```

```
end
```

```
nodes = collect(keys(node_dict)) # List of nodes
```

```
# Convert arc_df to Arc Information
```

```
#-----
```

```
struct Arc # Arc struct
```

```
  name::String
```

```
  fromto::Tuple{String, String}
```

```
  trans_cost::Float64
```

```
  trans_time::Float64
```

```
end
```

```
arc_dict = Dict() # dictionary of arcs -> Arc struct
```

```
for df_arc in eachrow(arc_df) # add arcs to arc_dict
```

```
  arc_id = df_arc.arc_id
```

```
  node_from = df_arc.from_node
```

```

    node_to = df_arc.to_node
    trans_cost = df_arc.cost
    trans_time = df_arc.time_cost
    arc = Arc(arc_id, (node_from, node_to), trans_cost, trans_time)
    arc_dict[(node_from, node_to)] = arc
end

arcs = collect(keys(arc_dict)) # List of arcs of network
incoming = Dict() # dictionary of node -> nodes incident to the node
outgoing = Dict() # dictionary of node -> nodes incident from the node
for (node_from, node_to) in arcs # add nodes to incoming and outgoing
    push!(get!(incoming, node_to, []), node_from)
    push!(get!(outgoing, node_from, []), node_to)
end

# Convert bom_df to Bill of Material (BOM) Information
#-----
struct Component # Component struct
    comp_id::String
    comp_type::String
    repair_difficulty::Int
    repair_cost::Float64
    repair_time::Float64
    subcomps::Vector{Tuple{String, Int}}
end
BOM = Dict() # dictionary of component -> Component struct
for bom_item in eachrow(bom_df) # build dictionary
    comp_id = bom_item.parent_component_id
    comp_type = bom_item.parent_component_class
    comp_repair_difficulty = bom_item.parent_component_repair_difficulty
    comp_repair_cost = bom_item.parent_component_cost
    comp_repair_time = bom_item.parent_component_repair_time
    comp_repair_list = []
    sub_bom = subset(bom_df, :parent_component_id => ByRow(==(comp_id)))
    for sub_comp in eachrow(sub_bom)
        sub_comp_id = sub_comp.child_component_id
        sub_comp_qty = sub_comp.quantity_required
        push!(comp_repair_list, (sub_comp_id, sub_comp_qty))
    end
    comp = Component(comp_id, comp_type, comp_repair_difficulty,
        comp_repair_cost, comp_repair_time, comp_repair_list

```

```

    )
    BOM[comp_id] = comp
end
part_bom = subset(bom_df, :child_component_class => ByRow(=="part")) # add parts components to BOM
for bom_item in eachrow(part_bom)
    part_id = bom_item.child_component_id
    part_type = "part"
    part_repair_difficulty = 0
    part_repair_cost = 0
    part_repair_time = 0
    part_repair_list = []
    part = Component(part_id, part_type, part_repair_difficulty,
                     part_repair_cost, part_repair_time, part_repair_list)
    BOM[part_id] = part
end
components = collect(keys(BOM)) # List of components in BOM
subcomp_dict = Dict() # dictionary of comonents -> List of subcomponents
for comp in components
    subcomp_bom = subset(bom_df, :child_component_id => ByRow(==(comp)))
    subcomp_for_lst = []
    for subcomp_for in eachrow(subcomp_bom)
        push!(subcomp_for_lst, (subcomp_for.parent_component_id, subcomp_for.quantity_required))
    end
    subcomp_dict[comp] = subcomp_for_lst
end

# Convert demand_df to Demand Information
#-----
timesteps = 4
struct Demand # Demand struct
    name::String
    loc::String
    component::String
    total_qty::Int
    time_schedule::Vector
end
demands = Dict() # dictionary of each node and component pair -> Demand struct
for df_demand in eachrow(demand_df) # build dictionary
    demand_id = df_demand.demand_id

```

```

demand_node = df_demand.node_id
demand_comp = df_demand.component_id
total_qty = df_demand.quantity
time_schedule = []
for slice in names(df_demand, r"^time_slice_")
    push!(time_schedule, df_demand[slice])
end
timesteps = length(time_schedule)
demand = Demand(demand_id, demand_node, demand_comp, total_qty, time_schedule)
demands[(demand_node, demand_comp)] = demand
end

# Convert node_inv_df to Node Inventory Information
#-----
struct Stock # Stock struct
    loc::String
    component::String
    qty::Int
end
inventory = Dict() # dictionary of each node and component pair -> Stock struct
for df_stock in eachrow(node_inv_df) # build dictionary
    stock_node = df_stock.node_id
    stock_comp = df_stock.component_id
    qty = df_stock.quantity
    stock = Stock(stock_node, stock_comp, qty)
    inventory[(stock_node, stock_comp)] = stock
end
M = sum(node_inv_df.quantity) # generate Big-M for Flow

# Convert comp_mult_df to Component Multiplier Information
#-----
struct Mults # Mult struct
    loc::String
    component::String
    repair_cost_mult::Float64
    repair_time_mult::Float64
end
mult_dict = Dict() # dictionary of each location and component -> Comp_Mult struct

```

```

for df_mults in eachrow(mult_df) # build dictionary
    mults_node = df_mults.node_id
    mults_comp = df_mults.component_id
    repair_cost_mult = df_mults.repair_cost_multiplier # change column name to repair_cost_mult
    repair_time_mult = df_mults.repair_time_multiplier # change column name to repair_time_mult
    mults = Mults(mults_node, mults_comp, repair_cost_mult, repair_time_mult)
    mult_dict[(mults_node, mults_comp)] = mults
end

```

```

In [32]: # Parameters
lambda_time = 1 # tradeoff parameters
B = 500         # backorder penalty
C = 1 / 10      # marginal transport price ratio

# Model
m = Model(Gurobi.Optimizer)

# Variables
@variable(m, x[arc in arcs, comp in components, timestep in 1:timesteps]
           >= 0, Int) # flow x of component through arc
@variable(m, y[arc in arcs, timestep in 1:timesteps], Bin) # y gives if flow through arc
@variable(m, r[node in nodes, comp in components, timestep in 1:timesteps]
           >= 0, Int) # repair r of component at node
@variable(m, s[node in nodes, comp in components, timestep in 0:timesteps]
           >= 0, Int) # store s of component at node
@variable(m, b[node in nodes, comp in components, timestep in 0:timesteps]
           >= 0, Int) # backorder b of component at node

# Constraints
## Flow Conservation
for comp in components, node in nodes, timestep in 1:timesteps
    inflow = sum(x[(node_from, node), comp, timestep] # flow of component into node
                 for node_from in incoming[node])
    outflow = sum(x[(node, node_to), comp, timestep] # flow of component out of node
                 for node_to in outgoing[node])
    demand = get(demands, (node, comp), Demand("", "", "", 0, # demand of component present at node

```



```

        zeros(Int, timesteps))).time_schedule[timestep] +
        b[node, comp, timestep - 1]                # previos backorders
backorders = b[node, comp, timestep]                # backorders of component at node
stock_initial = s[node, comp, timestep - 1]         # initial stock of component at node
stock_final = s[node, comp, timestep]               # final stock of component at node
repairs = r[node, comp, timestep]                   # repairs of component at node
consumed = sum(qty * r[node, comp_repaired, timestep] # component consumed for repairs at node
              for (comp_repaired, qty) in subcomp_dict[comp]; init=0)
@constraint(m, stock_initial + inflow + repairs + backorders ==
            stock_final + outflow + consumed + demand)
end

## Cannot Repair into Part Components
for comp in components
    if BOM[comp].comp_type == "part"
        for node in nodes, timestep in 1:timesteps
            @constraint(m, r[node, comp, timestep] == 0)
        end
    end
end

## Limit Node Repairs to Repair Capacity Hours
for node in nodes, timestep in 1:timesteps
    @constraint(m, sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
                      * BOM[comp].repair_time * r[node, comp, timestep] for comp in components)
               <= node_dict[node].repair_capacity)
end

## Cannot Complete Repairs of Difficulty at Nodes with Insufficient Repair Capability
for node in nodes, comp in components
    if BOM[comp].repair_difficulty > node_dict[node].repair_capability
        for timestep in 1:timesteps
            @constraint(m, r[node, comp, timestep] == 0)
        end
    end
end
end

```

```

## Initial Storage is Inventory
for node in nodes, comp in components
    @constraint(m, s[node, comp, 0] == get(inventory, (node, comp), Stock("", "", 0)).qty)
end

## Backorders Must be Fulfilled
for node in nodes, comp in components, timestep in [0, timesteps]
    @constraint(m, b[node, comp, timestep] == 0)
end

## Transport Decision y Constraint
for arc in arcs, comp in components, timestep in 1:timesteps
    @constraint(m, x[arc, comp, timestep] <= M * y[arc, timestep])
end

# ## No Early Fulfillment at Customer Nodes
# for node in nodes, comp in components, timestep in 1:timesteps
#     if node_dict[node].node_type == "customer"
#         @constraint(m, s[node, comp, timestep] == 0)
#     end
# end

# Objective
@objective(m, Min,
    sum(arc_dict[arc].trans_cost * y[arc, timestep]
        for arc in arcs, timestep in 1:timesteps) + # transport cost base
    sum(C * arc_dict[arc].trans_cost * x[arc, comp, timestep]
        for arc in arcs, comp in components, timestep in 1:timesteps) + # transport cost marginal
    sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_cost_mult
        * BOM[comp].repair_cost * r[node, comp, timestep]
        for comp in components, node in nodes, timestep in 1:timesteps) + # repair cost

```

```

sum(B * b[node, comp, timestep]
    for comp in components, node in nodes, timestep in 1:timesteps) + # backorder cost
lambda_time * (sum(arc_dict[arc].trans_time * y[arc, timestep]
    for arc in arcs, timestep in 1:timesteps) + # transport time
sum(get(mult_dict, (node, comp), Mults("", "", 1., 1.)).repair_time_mult
    * BOM[comp].repair_time * r[node, comp, timestep]
    for comp in components, node in nodes, timestep in 1:timesteps)) # repair time
)

# Solve
set_silent(m)
optimize!(m)

```

Set parameter Username

Set parameter LicenseID to value 2807856

Academic license - for non-commercial use only - expires 2027-04-15

```

In [33]: # Output
println("Optimal Objective Value: ", round(objective_value(m), digits=2))
println()

## Flow Output
flow_output_df = DataFrame(timestep = Int[],
                           comp_id = String[],
                           node_from = String[],
                           node_to = String[],
                           qty = Int[]
                           )
for comp in components, (node_from, node_to) in arcs, timestep in 1:timesteps # store optimal flow in dataframe
    val = value(x[(node_from, node_to), comp, timestep])
    if val > 1e-3
        push!(flow_output_df, (timestep = timestep,
                                comp_id = comp,
                                node_from = node_from,
                                node_to = node_to,
                                qty = val))
    end
end

```

```
sort!(flow_output_df, [:timestep, :comp_id, :node_from, :node_to])
println("Optimal Flow Schedule:") # output optimal flow dataframe
println(flow_output_df)
println()

## Repair Output
repair_output_df = DataFrame{timestep = Int[],
                             comp_id = String[],
                             node_id = String[],
                             qty = Int[]
                             }
for comp in components, node in nodes, timestep in 1:timesteps # store optimal repair in dataframe
    val = value(r[node, comp, timestep])
    if val > 1e-3
        push!(repair_output_df, (timestep = timestep,
                                comp_id = comp,
                                node_id = node,
                                qty = val)
        )
    end
end
sort!(repair_output_df, [:timestep, :comp_id, :node_id,])
println("Optimal Repair Schedule:") # output optimal repairs
println(repair_output_df)
println()

## Backorder Output
backorder_output_df = DataFrame{timestep = Int[],
                                comp_id = String[],
                                node_id = String[],
                                qty = Int[]
                                }
for comp in components, node in nodes, timestep in 1:timesteps # store optimal backorders in dataframe
    val = value(b[node, comp, timestep])
    if val > 1e-3
        push!(backorder_output_df, (timestep = timestep,
                                    comp_id = comp,
                                    node_id = node,
```

```
                qty = val)
            )
    end
end
sort!(backorder_output_df, [:timestep, :comp_id, :node_id,])
println("Optimal Backorder Schedule:") # output optimal backorders
println(backorder_output_df)
```

Optimal Objective Value: 2431.5

Optimal Flow Schedule:

26x5 DataFrame

Row	timestep	comp_id	node_from	node_to	qty
	Int64	String	String	String	Int64
1	1	LRU_1	DEPOT_1	REGWH_1	2
2	1	LRU_1	LOCAL_2	CUST_2	2
3	1	LRU_1	REGWH_1	LOCAL_2	2
4	1	LRU_1	SPEC_3	DEPOT_1	1
5	1	LRU_1_1	DEPOT_1	SPEC_3	1
6	1	LRU_4_1	DEPOT_1	SPEC_3	1
7	1	LRU_4_2	DEPOT_1	SPEC_3	1
8	1	LRU_5	DEPOT_1	REGWH_1	4
9	1	LRU_5	LOCAL_2	CUST_1	3
10	1	LRU_5	LOCAL_2	CUST_3	1
11	1	LRU_5	REGWH_1	LOCAL_2	4
12	1	LRU_5	SPEC_3	DEPOT_1	3
13	1	LRU_5_2	DEPOT_1	SPEC_3	3
14	1	LRU_6	DEPOT_1	REGWH_1	3
15	1	LRU_6	LOCAL_2	CUST_2	3
16	1	LRU_6	REGWH_1	LOCAL_2	3
17	1	LRU_6	SPEC_2	DEPOT_1	3
18	1	LRU_6_2	DEPOT_1	SPEC_2	2
19	2	LRU_1	DEPOT_1	REGWH_2	2
20	2	LRU_1	REGWH_2	LOCAL_3	2
21	2	LRU_1	SPEC_3	DEPOT_1	2
22	2	LRU_4	DEPOT_1	REGWH_2	1
23	2	LRU_4	LOCAL_3	CUST_4	1
24	2	LRU_4	REGWH_2	LOCAL_3	1
25	2	LRU_4	SPEC_3	DEPOT_1	1
26	3	LRU_1	LOCAL_3	CUST_3	2

Optimal Repair Schedule:

7x4 DataFrame

Row	timestep	comp_id	node_id	qty
	Int64	String	String	Int64
1	1	LRU_1	DEPOT_1	1
2	1	LRU_1	SPEC_3	1
3	1	LRU_5	DEPOT_1	1

4		1	LRU_5	SPEC_3	3
5		1	LRU_6	SPEC_2	3
6		2	LRU_1	SPEC_3	2
7		2	LRU_4	SPEC_3	1

Optimal Backorder Schedule:

0x4 DataFrame

Row	timestep	comp_id	node_id	qty
	Int64	String	String	Int64

Conclusion

Our results demonstrate several ways to model an aircraft repair network under increasing levels of realism. Level 0 established the basic repair-and-shipping flow. Level 1 introduced location-dependent repair skill and cost. Level 2 added repair time and capacity limits. Level 3 extended the model across multiple time periods with changing demand and the option to backlog work, while Level 3.5 allowed batched shipping. Using the final model, the system was able to satisfy all 11 units of customer demand across four time periods in our simplified example.

Across the Level 2 and Level 3 models, we found that the model chose to perform nearly all repairs at either the central depot or a specialized shop. This reflects an important cost-time tradeoff. Lower-cost facilities, such as central depots, are attractive when the objective primarily emphasizes cost minimization. However, specialized shops become more valuable when repair speed is weighted more heavily. This tradeoff was especially clear in the Pareto time-versus-cost analysis: as the weight placed on time increased, the model shifted more repairs toward specialized shops despite their higher repair costs.

We also observed that the optimal objective value generally increased as simplifying assumptions were removed. Because this is a minimization problem, a higher objective value represents a less ideal but more realistic solution. Part of this increase comes from adding time costs to the objective, but it also reflects a broader modeling lesson: simplifying assumptions can significantly under-predict the true cost of a system. Early in the course, we discussed the idea that *all models are wrong, but some are useful*. Our project's evolution reflects this principle. As our model moved closer to real-world repair logistics, the solution became more expensive (and hopefully less wrong), but also more informative and useful for decision-making.

Future Work

There are several future directions that could build on this work. Our model focused primarily on capturing how repair logistics evolve over time. However, to further improve the usefulness of the model, future work could better represent shipping and movement costs. For example, different items may have different transportation costs, and each route

may offer multiple shipping methods with distinct cost, time, and capacity tradeoffs. Incorporating these details would significantly improve the model's predictive capability. Another important extension would be modeling how to select the *best* option when demand cannot be fully satisfied. In this project, we assumed that enough parts and repair capacity were available to fulfill all demand. However, this may not hold in real-world situations. If the system lacks sufficient parts, repair hours, or transportation capacity, the model would need to decide which repairs to complete immediately and which to delay. This would allow the model to support decision-making under scarcity, where the goal is not only to minimize cost and time, but also to prioritize the most operationally important repairs.

AI Acknowledgment

Codex [3], powered by GPT 5.4, was used to generate and refine the Python scripts used to generate the input data and visualizations. We went through an iterative process that consisted of a mix of prompting codex, manually inspecting the Python code, and implementing further code refinements by hand. Codex was not utilized in the creation of Julia code or any model-building.

ChatGPT [4] was used to improve the grammar and spelling of this work as well as debug some of the above Julia code.

If desired, a record of the major edits to this work can be viewed online in our [Github repository](#).

Citations

1. Sherbrooke, C. C. (2004). *Optimal inventory modeling of systems* (2nd ed.). Kluwer Academic. (International Series in Operations Research & Management Science).
2. Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual*. 2026. <https://www.gurobi.com> (accessed April 29, 2026).
3. OpenAI. Codex-GPT.5.3. April 2026. Large language model. <https://openai.com/>
4. OpenAI. ChatGPT. April 2026. Large language model. <https://chat.openai.com/>